

Department of Physics, Astronomy and Mathematics

MSc Data Science Professional Team Project

7PAM2033

GROUP FINAL REPORT

Agentic Multi-Source Adverse Drug Event Detection: An Autonomous Sentiment Analysis System with Cross-Domain Generalisation and Explainable Clinical Triage

Team Members

Student name	Student SRN number
Chisom Francis Ofonagoro	24101661
Sai Charan Ganagani	24142673
Muhammad Bhatti	23100117
Chinedu Samuel Umeh	24137199
Renjie Fu	24142372
Deepak Kottakkal Murali	24140337

Team GitHub link

: <https://github.com/Professional-Team-Project-Group-13/Week-11-Team-13>

Supervisor: Dr. Mykola Korabel

Date Submitted: 22nd July 2026

Word Count: 8,275 words

DECLARATION STATEMENT

Any material derived or quoted from published or unpublished work of other persons has been duly acknowledged. (Ref. UPR AS/C/6.1, section 7 and UPR AS/C/5, section 3.6)

Human participants were not used in this MSc Project.

I have read the detailed guidance on academic integrity, misconduct and plagiarism at [Assessment Offences and Academic Misconduct](#) and understand the University process for dealing with suspected cases of academic misconduct and the possible penalties, which could include failing the project or course.

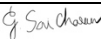
Permission is given for the report to be made available on module websites provided the source is acknowledged.

Declaration of Use of AI

[Delete the Yes or No as appropriate]

- **Yes/No** AI tools have been used in the preparation of this project and report
 - **No** To generate initial ideas.
 - **No** To develop code.
 - **No** To provide literature suggestions and summaries.
 - **No** To provide suggestions to improve the report content.
 - **No** To edit spelling and grammar, and to proofread the report.
 - **No** Other (provide details):

Note: Copying text generated by AI into the report is academic misconduct.

Student name	Student SRN number	Signature	Date
Chisom Francis Ofonagoro	24101661		20/07/2026
Sai Charan Ganagani	24142673		21/07/2026
Muhammad Bhatti	23100117		21/07/2026
Chinedu Samuel Umeh	24137199		20/07/2026
Renjie Fu	24142372		20/07/2026
Deepak Kottakkal Murali	24140337		21/07/2026

Acknowledgments

We thank our project module leader and our supervisor for their guidance throughout the project. This work would not have been possible without several open data and software resources, which we gratefully acknowledge.

Data: Formal-domain data were obtained from DailyMed, curated by the U.S. National Library of Medicine (NLM). Informal-domain data were obtained from the CSIRO Adverse Drug Event Corpus (CADEC); we thank CSIRO for making the corpus available for research and note that CADEC is used under its research licence and is not redistributed in this repository.

Software: Our implementation relies on scikit-learn, PyTorch, the Hugging Face Transformers and Tokenizers libraries, the pre-trained BioBERT weights released by Lee et al. (2020), FAISS for similarity search, LIME and SHAP for explainability, and pandas, NumPy, Matplotlib and Plotly for data handling and visualisation. The interactive application was built with Streamlit and Flask

Abstract

Adverse drug events are a major cause of avoidable harm, and the evidence for them is written in two very different registers: formal regulatory product labelling and informal patient narrative. SafetyNet AI asks whether a single detection system can serve both. We harmonise 3,077 records from DailyMed (1,831 formal) and CADEC (1,246 informal) into one binary ADE-detection corpus and evaluate six models of increasing complexity: TF-IDF with logistic regression, random forest and XGBoost; a Bi-LSTM with attention; an SVM over frozen BERT embeddings; and a fine-tuned BioBERT, together with a stacked ensemble.

Within a single register, the task is close to saturated. Every model exceeds 0.97 binary F1 on the pooled same-domain test set, and the TF-IDF baseline differs from fine-tuned BioBERT by 0.002 F1 on four records on a 451-record test set, indicating that the corpus is strongly lexically separable rather than that the models are unusually capable. Evaluated per register, the informal partition is markedly harder (macro-F1 0.715–0.875) than the formal one, whose LOINC-derived labels are close to deterministic.

Across registers, the picture reverses completely. Under a hash-verified leakage-free protocol in which each model is trained on one domain and evaluated on the other, every model collapses on the Formal-to-Informal transfer. All six score between 0.0824 and 0.1089 macro-F1, with balanced accuracy between 0.493 and 0.504 against a chance level of 0.500, and three of the six predict no positive case whatsoever on a target partition that is 91.0% positive. The fine-tuned BioBERT, the strongest in-domain model in the study and the best on the harder informal partition at 0.8745 macro-F1, is the only model whose Formal-to-Informal ROC-AUC falls below chance, at 0.492.

Biomedical pre-training bought in-domain accuracy and no transfer at all.

The central contribution is the explanation rather than the ranking. Models inherit the class prior of the register they were trained on: formally trained models predict ADE for between 0.0% and 2.9% of a target that is 91.0% positive, while informally trained models predict ADE for up to 99.3% of a target that is 39.7% positive. A label-free correction that matches the predicted positive rate to the known target prevalence separates threshold shift from representational failure, and the two directions prove to fail for different reasons. Informal-to-Formal is recoverable: the Bi-LSTM ranks that transfer at ROC-AUC 0.995 and recovers from 0.397 to 0.991 macro-F1 under correction, with the six-model mean rising from 0.453 to 0.747. Formal-to-Informal is not: ranking there has itself collapsed to between 0.492 and 0.664 ROC-AUC, and correction lifts the mean only from 0.091 to 0.512. The residual gap is explained by lexical divergence; the two registers share only 14.3% of their top 300 vocabulary.

The deployed system - a fine-tuned BioBERT trained on the pooled corpus, with LIME and SHAP attribution, FAISS retrieval of labelled precedents, and an agentic triage layer that escalates uncertain or severe cases to a human reviewer - is in-domain for both registers by construction and is not subject to this failure. The cross-domain results define its operating boundary rather than its expected performance. We conclude that cross-register ADE detection should be treated as a prior-correction and calibration problem before it is treated as a representation-learning problem, and that reporting binary F1 alone on imbalanced clinical corpora conceals exactly this failure.

Contents

Acknowledgments	4
Abstract.....	5
1. Introduction	9
1.1 Background and related work	9
1.2 Research question	10
1.3 Objectives	10
2. Data	11
2.1 Choice of datasets and background	11
2.3 Exploratory data analysis	12
2.4 Pre-Processing	13
2.5 Feature Engineering	13
2.6 Data Ethics Statement	14
3. Methodology	14
3.1 Task formulation and metrics	15
3.2 Classical baselines	15
3.3 Bi-LSTM with attention.....	15
3.4 SVM-BERT	16
3.5 BioBERT (fine-tuned)	16
3.6 Ensemble (stacked meta-learner)	16
3.7 Explainability and retrieval	16
3.8 Agentic triage layer	17
3.9 Cross-domain protocol	17
3.10 Application demonstration	18
3.11 Testing, reproducibility and CI/CD	18
3.12 Hyperparameter optimisation and training diagnostics	18
4. Results and Analysis.....	18
4.1 Same-domain model comparison	19
4.2 Cross-domain generalisation: The core result	19
4.2.1 Prior correction: separating threshold shift from representational failure	22
4.3 Comparison with the literature	23
4.4 Explainability results.....	24
4.5 Application demonstration analysis.....	25
4.6 Answering the research question.....	26
4.7 Applications and use-cases.....	26

4.8 Limitations	27
5. Conclusions and Future Work	28
5.1 Conclusions	28
5.2 Future work	28
References	30
Appendices	32
Appendix A — Repository structure and code	32
Appendix B — Provenance	56
Appendix C — Team roles and project management.....	57
Appendix D — Tables:	57

1. Introduction

1.1 Background and related work

An adverse drug event (ADE) can cause significant damage as a result of a medication error. It is one of the most common sources of preventable injuries in healthcare, resulting in hospitalizations, extended hospital stays, and deaths globally (Gurulingappa et al., 2012). The discipline of pharmacovigilance, which is concerned with detecting and preventing harms, has traditionally relied on immediate reporting systems and analysis of regulatory documents.

Two key changes are happening in this field. On the one hand, the number of free-text reports of safety issues has become too high for human analysis. On the other hand, patients share their personal experience of using medications in their own words in forums and social media. As a result, we have a very abundant, albeit noisy, informal signal that complements the formal safety reports (Karimi et al., 2015; Sarker and Gonzalez, 2015). These two data types are noticeably different in style. The official drug labels, including those published by the FDA and available on DailyMed, use formal language, have structured headings, and use professional register. Patient comments (in the CADEC dataset) use vernacular language, include typos, carry emotional coloring, and describe symptoms.

A detector that works well on one data type does not necessarily perform equally well on another data type due to different surface properties. Cross-domain generalization is thus the main challenge in this project. Modern NLP research provides advanced methods to address this challenge. Vaswani et al. (2017) developed the transformer architecture, while Devlin et al. (2019) took it further with BERT model, which captures not only lexical surface properties but also the semantic meaning of text. Lee et al. (2020) applied the same approach to the biomedical domain with BioBERT, pre-trained on PubMed. The trust in these systems grew due to the introduction of explanations for the predictions by Ribeiro et al. (2016) and Lundberg and Lee (2017), who proposed LIME and SHAP. Large-scale nearest neighbor search became possible after the release of Johnson, Douze, and Jégou (2019) algorithm for FAISS. All these innovations allow not only to classify the case but also to explain the decision, which is critical for medical applications.

In this work, these modules are combined into a unified cross-domain ADE detection pipeline equipped with the agentic triage component. Instead of emphasizing high accuracy, the research brings to the forefront one of the most important clinical questions: beyond accuracy, is there a way for the model to demonstrate that its performance is authentic, transferable, and sufficiently interpretable? Automated detection of ADE has passed through three generations. Lexical and rule-based approaches are simple but fragile: they usually fail to consider linguistic variations and typical mistakes made by patients. Classical machine learning techniques transform documents into TF-IDF vectors and apply methods such as logistic regression, SVMs, or tree-based ensembles. Gurulingappa et al. (2012) and Sarker and Gonzalez (2015) confirmed that these algorithms have been proven robust and reliable as lexically driven techniques, and our experiments confirm the same result. Deep learning era has brought sequence models. LSTM neural networks were invented by Hochreiter and Schmidhuber (1997). Bahdanau, Cho, and Bengio (2015) implemented an attention mechanism which we use in our Bi-LSTM. Vaswani et al. (2017) introduced the transformer model. Its biomedical adaptation, BioBERT has been done by Lee et al. (2020). LIME and SHAP explainability methods have been proposed by Ribeiro et al. (2016) and Lundberg and Lee (2017): they are recognized as necessary in order to deploy deep models in the clinical environment.

There are two main reasons for this research. First, most of ADE studies are performed on single registers, while the works of Weissenbacher et al. (2019) show that there is not enough generalization between different registers, and cross-domain transfer is underexplored. Second, the improvements of headline metrics on imbalanced pharmacovigilance datasets often raise doubts

about the authenticity of these improvements and whether they represent real learning or simply reflect the specifics of these datasets. These problems are solved by SafetyNet AI in a rigorous way.

1.2 Research question

How effectively do machine learning classifiers identify and classify adverse drug events using disparate data sets, ranging from structured FDA regulatory documents to unstructured patient reports? How much generalization can be achieved when the classifier was trained using one data modality (formal labels) and evaluated on another one (patient narratives)?

There is an empirical side to this question. One needs to analyze the transferability of performance between different data modalities, as well as compare models of different complexities with each other. There is also a question about the possibility of explanation and contextualization of predictions made by the model. This point is especially important due to the fact that while a model works well within one data modality, it may show poor performance on another one. Minor differences in accuracy scores may indicate either generalization or memorization of a certain writing style.

Term definition: this project is referred to as an "autonomous sentiment analysis system." Sentiment analysis, in this case, means the application of natural language processing techniques in order to classify the patient-generated subjective text. This is not a three-way classification of sentiment but rather a two-way ADE / no-ADE classification task.

1.3 Objectives

Consistent with the project specification, the work is structured according to five increasingly broad objectives:

1. Data integration: Creation of a consistently aligned ADE database via synthesis of DailyMed (formal) and CADEC (informal) data sources, including full pre-processing (cleaning, tokenization, label alignment) and a stratified 70/15/15 data split for both domains.
2. Comparative model analysis: Design and evaluation of at least five increasingly complex models: baseline models based on TF-IDF features (logistic regression, random forest, XGBoost), custom Bi-LSTM with attention mechanism, hybrid SVM-BERT model, fine-tuned BioBERT model, and ensemble meta-learner. Such a configuration allows for performing comparative analysis of simple and complex approaches.
3. Generalization between domains: Investigation of performance differences when training models on one domain and testing them across all domains using a specially crafted transfer matrix, which prevents data leakage. Analysis of results is needed instead of uncritical adoption of results.
4. Explainable AI and clinical triage: Generation of explanations of prediction using approaches like LIME, SHAP, attention, and FAISS-based case search. Predictions are mapped to four severity levels (none/mild/moderate/severe) supplemented by a routing mechanism based on confidence.
5. Application development: Development of an interactive application emphasizing detection, explainability, cross-domain analysis, batch processing, and maintaining explicitly stated data ethics and governance, acceptable for clinical decision support. The specification provides quantitative targets in terms of three levels (baseline/good/excellent) and five performance indicators related to

model performance, cross-domain generalization, explainability quality, application quality, and cross-domain robustness. Results reported in Section 4 exceed F1 goals for domain-internal models, but the cross-domain degradation goal became the main result of this project instead of the checkbox. Section 4.6 revisits the research question in the light of the results.

2. Data

2.1 Choice of datasets and background

The datasets are based on text data from two different sources that have been annotated for adverse event mentions. Two trustworthy, publicly available sources were used.

DailyMed (formal domain): DailyMed is an authoritative source of FDA-approved structured product labeling provided by the U.S. National Library of Medicine (2026). Labels are provided as SPL XML files and are updated regularly. In this analysis, the April 2026 monthly release was used. Each label is made up of professionally written sections which include adverse reactions, warnings, contraindications, etc. The data from DailyMed is in the public domain; hence, it can be used in reproducible research.

CADEC (informal domain): CSIRO Adverse Drug Event Corpus (CADEC) consists of posts in medical forums that are written by patients. The posts are annotated for drugs, adverse effects, symptoms, and diseases (Karimi et al., 2015). CADEC was sourced from the AskaPatient forum, which is provided for research purposes by CSIRO (Australia). The language is informal and varied as opposed to the formal language in the DailyMed.

To standardize the two data sources, each dataset was mapped to one schema that had one binary label. Evidence positive (ADE) and negative (no-ADE) in the formal domain were identified by using LOINC section codes (positive: 34084-4 for adverse reactions; negative: 34067-9 and 34068-7 for indications/usage sections). The span annotation in CADEC was mapped to the record level. The resulting unified schema contains the following fields: text, text_processed, label, severity, severity_source, domain, group_id, and word_count_processed. A provenance record (random seed 42, pull timestamp 16 June 2026) was kept.

The acquisition pipeline was managed through the team's GitHub manager in order to provide all the members with synchronized information. DailyMed labels were downloaded from the NIH SPL service, while CADEC was sourced from the CSIRO repository in accordance with CC BY 3.0 license. Checksums for integrity validation were performed through SHA-256, raw data was left out of the version control system, processed datasets were logged with Git LFS, and checkpoints were backed up on institutional cloud storage. In this way, a simple yet valid data lineage from the source to the result was provided.

The aligned corpus consists of 3,077 samples. Table 1 below gives the breakdown of its composition. The whole corpus shows a moderate positive attitude (60.2% ADE); nevertheless, this number hides a significant difference between the two sources. The former source shows a minority-positive attitude (39.2%), while the latter one is overwhelmingly positive (91.1%). This difference cannot be considered a negligible issue and should be taken into account since it is key to the interpretation of cross-domain results described in Section 4.

Partition	Records	ADE (1)	No-ADE (0)	ADE prevalence
Formal (DailyMed)	1,831	718	1,113	39.2%
Informal (CADEC)	1,246	1,135	111	91.1%
Full corpus	3,077	1,853	1,224	60.2%

Table 1. Composition of the harmonised cross-domain ADE corpus

In terms of modeling, the corpus was split into training, validation, and testing sets, maintaining the label balance in each case. The sizes and label balances of each domain are shown in Table 2. Domain-level imbalance is uniform throughout the three data splits. Notably, the same-domain test split maintains a 60.1% positive balance, but informal cases within the same domain show a 93.0% positive balance.

Split	Records	ADE prevalence	Formal (% ADE)	Informal (% ADE)
Train	2,156	60.0%	1,284 (38.9%)	872 (90.9%)
Validation	470	61.5%	283 (42.8%)	187 (89.8%)
Test	451	60.1%	264 (36.7%)	187 (93.0%)

Table 2. Stratified split sizes and per-domain class balance (source: train/val/test CSVs).

2.3 Exploratory data analysis

An exploratory analysis is carried out in six panels (see Figure 1 below). This analysis involves the modelling of the corpus and is used to inform the modelling and analysis further.

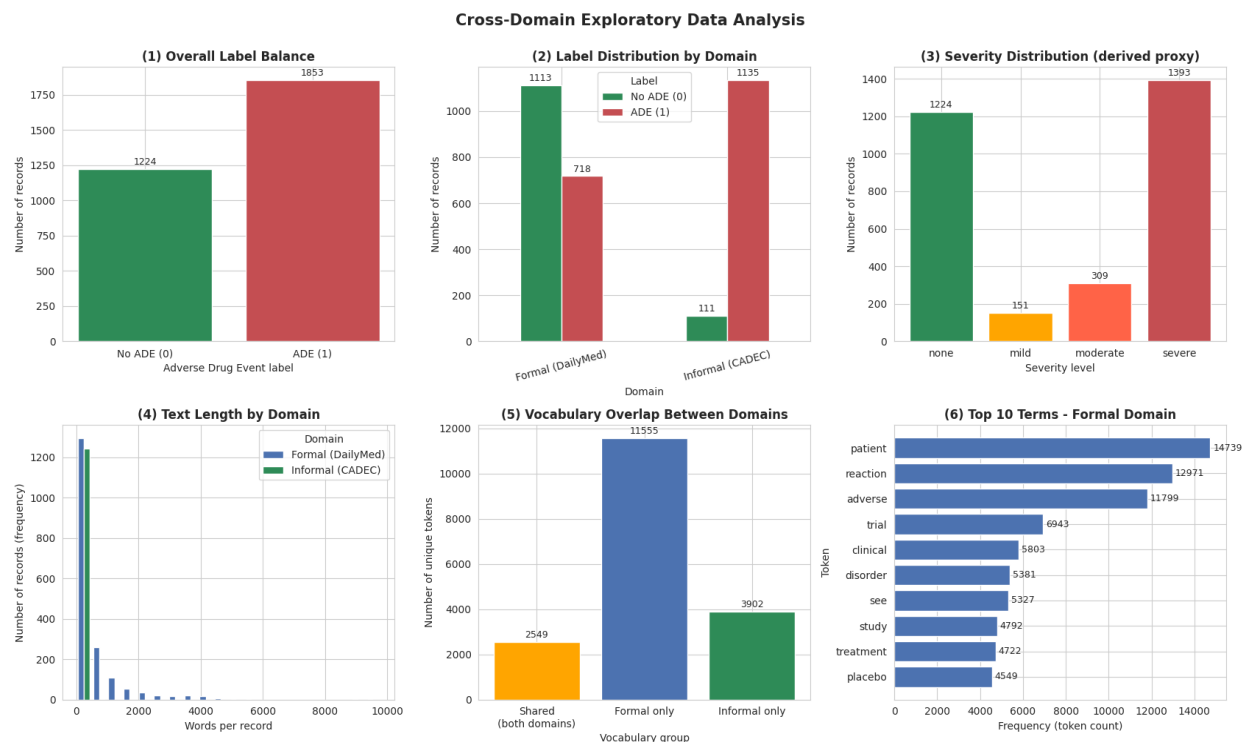


Figure 1. Cross-domain exploratory data analysis. Panels: (1) overall label balance; (2) label distribution by domain; (3) derived severity proxy; (4) text length by domain; (5) vocabulary overlap; (6) top terms in the formal domain.

Four key observations can be made. Firstly, there is noticeable variance in class balance across the domains (panels 1 and 2). The informal corpus is almost completely ADE-positive, which is expected to make the recall-oriented metric like F1 higher in any informal test set. Secondly, there is notable bimodality in the severity proxy illustrated in the third panel: most (1,224) of the proxies are "none," and others are "severe" (1,393). It needs to be emphasized that the proxy severity cannot be treated as a label but should be viewed as a heuristic one. Thirdly, text length varies depending on the register (panel 4): formal labels tend to be long and may exceed the word limit in length, while informal texts are shorter. Fourthly, vocabulary overlap between domains is low (panel 5): from the unique tokens, only 2,549 are common, whereas 11,555 are specific to formal text and 3,902 are specific to informal text. Formal top terms include patient, reaction, adverse, trial, and clinical (panel 6). This low cross-domain vocabulary overlap explains the challenge of transfer learning between domains. It is confirmed in section 4.2, where models based on this vocabulary show reduced performance in the other register.

2.4 Pre-Processing

Two parallel textual representations have been used. The lightly normalized field (`text_processed`) contains lower-casing, normalization of whitespaces and punctuation, and removal of XML/markup features. The field is needed for classical and TF-IDF approaches. The raw text field does not undergo any processing and is used for transformer models (BioBERT, SVM-BERT) since subword tokenizers are trained on natural language texts and excessive cleaning can spoil their casing and morphological structure, which will be used by the models. Deduplication removes exact duplicate entries, and the `word_count_processed` field is kept, allowing analysis based on text length. Importantly, all splits are done before model training, and vectorizers and scalers are generated only from the train set to avoid leakage of test information to training.

The needs for text processing were different for the two registers. Formal text from DailyMed has been parsed from the raw SPL XML without relevant labeling parts according to public-domain guidelines and with markup removal to get clean clinical text. Informal text from CADEC has been processed to remove URLs and non-standard characters. Misspellings from patients were corrected using Levenshtein distance against the clinical dictionary. The difference in processing methods highlights the main problem of the study – two domains represent different linguistic registers.

2.5 Feature Engineering

The document representation in classical models relies on TF-IDF vectors based on unigrams and bigrams with sub-linear term frequency weighting and limited vocabulary size. This design allows encoding strong signals from individual words and short sequences detected through exploratory data analysis (such as "adverse reaction" and "warning"). Feature engineering is not required in deep learning and transformer models. Bi-LSTM uses embeddings to build a sequence representation, while SVM-BERT and BioBERT provide pre-trained contextualized embeddings. The domain field enables cross-domain evaluation, while the `group_id` field allows grouped evaluation. The severity field serves purely heuristic purposes and is used exclusively for routing and visualization.

2.6 Data Ethics Statement

There are four ethical issues associated with the data management plan of the project, and each one is considered subsequently.

Does the data meet GDPR? No. DailyMed is the FDA regulatory public-domain documentation available under 21 CFR Part 20.61 and does not contain any personal data. CADEC is made available by CSIRO in a de-identified format, where all direct identifiers like patients' names, email addresses, IP addresses are removed by the curators of the dataset. No efforts of re-identification are applied. As the data in both sources is anonymized and does not contain personal data, UK GDPR and the Data Protection Act 2018 do not apply. The application confirms that it is only using the entered text in memory and records the actions, but not their contents.

Does the project satisfy the University's ethical standards? Yes. This work represents a secondary analysis of two publicly available datasets, without collecting any new data and involving human participants. Thus, according to the University of Hertfordshire's ethical considerations, no further approval is needed in view of the lack of new human data, lack of any identifiable individual, and academic research goals.

Do we have permissions to use the data? Yes. DailyMed is in the public domain and can be freely used for research purposes. CADEC is available under the Creative Commons Attribution 3.0 license for academic, non-commercial usage, as long as the original authors (Karimi et al., 2015) are cited. The citation is provided, and the corpus is not distributed directly; instead, instructions are provided for downloading the data legally.

Was the data collected ethically? Yes. Both datasets were assembled in compliance with the relevant standards. DailyMed is official regulatory documentation, while CADEC complies with CSIRO's research ethics principles. Both datasets are only used for the stated research purposes. Apart from the provenance issue, other risks of the project are covered. The severity attribute is clearly described as a heuristic proxy; class imbalance, which improves performance, is openly stated and analysed through per-class and ranking metrics. There is no risk of failing to detect the adverse event because of the triage step when a case is referred to human review when there is some uncertainty or severity. The system is designed as a decision-supporting tool, not a diagnostic one.

3. Methodology

The methodology follows the order of models with increasing levels of complexity. Every single model is evaluated based on the same metric definition of the incremental improvement in performance from the previous step. Such structure is intentional since this will allow us to check whether a more complex model outperforms the simpler one, and whether the additional level of complexity is justified.

Figure 2 illustrates the entire system architecture. The two sources of data produce a unified corpus and a stratified split. The series of models produces probability scores which are aggregated through the ensemble. Explainability, retrieval and triage layers are located above the architecture in the application. Cross-domain evaluation is explained further in Section 3.9.

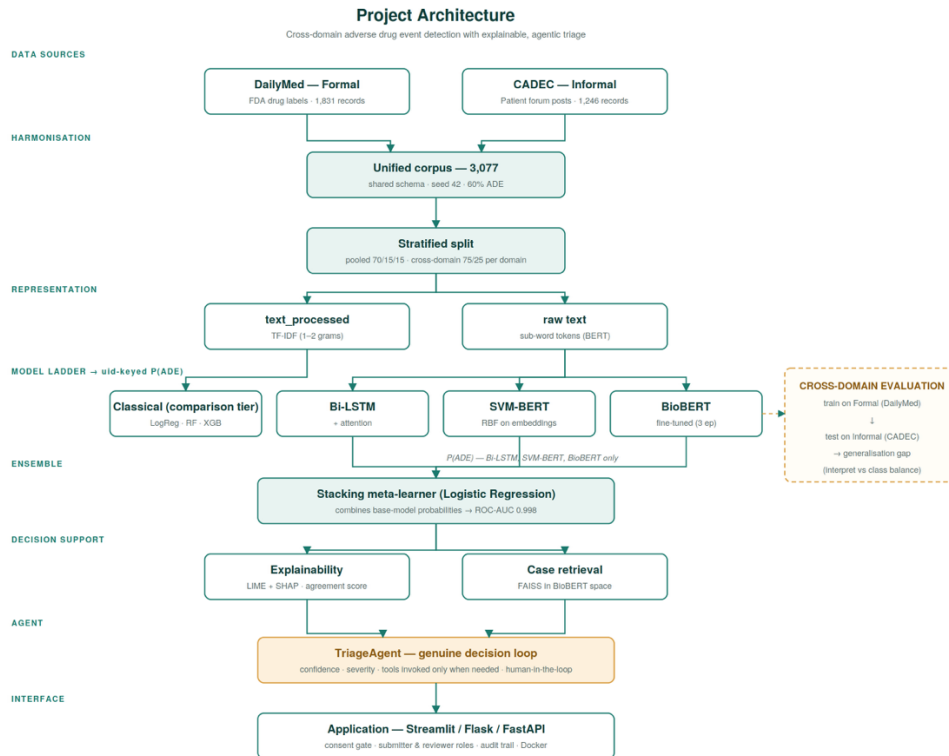


Figure 2. End-to-end system architecture.

3.1 Task formulation and metrics

This problem revolves around binary classification regarding the occurrence of adverse drug events (ADE) in individual records. All the models will be scored using a common metrics module (eval_utils.py) that computes the following measures: accuracy, precision, recall, F1 (binary; positive class = ADE), macro-F1, and ROC-AUC. Macro-F1 and ROC-AUC are included apart from F1 since, considering the imbalanced nature of the dataset, using F1 only would mean misleading evaluation of a classifier, which would always predict the majority class. ROC-AUC measures ranking ability without being dependent on the threshold value, whereas macro-F1 gives both classes equal importance. The threshold value is fixed at 0.5 unless stated otherwise.

3.2 Classical baselines

The three classical models that employ TF-IDF features set the benchmark for comparison, namely logistic regression, random forest, and XGBoost, implemented with scikit-learn (Pedregosa et al., 2011) and XGBoost (Chen and Guestrin, 2016). All three apply inverse-frequency class weighting to logistic regression and random forest via balanced class weights, XGBoost via a per-domain scale_pos_weight set to the training split's negative-to-positive ratio so that imbalance handling does not differ across the baseline tier and confound an experiment whose central mechanism is class-prior shift. They are highly efficient and easily interpretable. As shown in Section 4, these models turn out to be surprisingly strong in this corpus. They serve both as a benchmark and as an indication of the inherent difficulty of the task. Given that a linear model with word counts achieves such a high F1 score, the ADE signal must be primarily lexical.

3.3 Bi-LSTM with attention

The first neural network consists of a bidirectional LSTM (BiLSTM) network enhanced with an attention layer. The embedding layer represents tokens as vectors. The BiLSTM layers run the tokens

bidirectionally in order to consider the context of each token. An additive attention layer computes a weighted representation of tokens most likely indicative of an ADE in line with Bahdanau et al. (2015). The dense sigmoidal output predicts the probability of the occurrence of an ADE. Dropout regularization is applied along with the Adam optimization algorithm in binary cross-entropy loss. Early stopping based on validation loss is performed. The weights of the attention layer provide built-in explanations for how each prediction was made.

3.4 SVM-BERT

In SVM-BERT, the distinction between representation and classification is made. The context representation of each document, obtained through the pre-trained BERT encoder, is used to classify the document using a support vector machine with an RBF kernel. SVM-BERT is a very powerful and fast model that uses contextual representations without the cost of training the complete encoder. This is evident from the results, where it performs best in same-domain F1.

3.5 BioBERT (fine-tuned)

BioBERT is made up of BERT that is pre-trained on PubMed abstracts and PMC full texts, thus being aligned to the biomedical domain. BioBERT was fine-tuned end-to-end with the Transformers library (Wolf et al., 2020) for the binary ADE task in three epochs with the use of AdamW optimizer, a low learning rate, class-weighted loss due to class imbalance, and a classification head for the pooled representation. Additionally, BioBERT acts as the backbone for the explainability and retrieval modules. The embeddings provided by BioBERT define the semantic space where FAISS will search for precedent cases, and it forms the basis of models that are interpreted with LIME and SHAP.

3.6 Ensemble (stacked meta-learner)

The final model is built as a stacked ensemble (Wolpert, 1992). Out-of-fold predictions of probability scores of Bi-LSTM, SVM-BERT, and BioBERT models are exported as CSV files keyed on record id, and the scores are merged using record ids as the key. They act as input features for a logistic regression meta-learner, which will decide the weight of each base model. Using stacking on probabilities instead of hard labels enables us to utilize the calibrations of the base models. It is shown in Section 4 that such an ensemble reaches the highest ROC-AUC across all models but shares the same thresholded F1 with the best model.

Implementation and Tuning. The TF-IDF baselines were implemented using unigram and bigram with sub-linear scaling and bounded vocabulary. The regularization strength (logistic regression) and number of trees and depth (random forest and XGBoost) were tuned using the validation set. The Bi-LSTM used bidirectional LSTM layers, Bahdanau attention, and dropout 0.3, as well as Adam optimization and early stopping based on validation loss. BioBERT was fine-tuned for three epochs using AdamW with a linear warm-up and a learning-rate range of $2e-5$ to $5e-5$, sufficient maximum sequence length, and class-balanced binary cross-entropy loss. In the model selection stage, only the validation set is used, and the test set is used only once to produce final results in Section 4. All experiments used a fixed random seed of 42.

3.7 Explainability and retrieval

Explanation framework description: The stacked ensemble is the top-performing module of the system but is not an object of the explanation itself since the meta-learner works with probabilities of the base models and not tokens making token attribution impossible. The explanation process is focused on BioBERT that plays the role of a representational and interpretive core of the system. It was done deliberately since the ensemble predicts risk and BioBERT has F1 score differing from the ensemble's by at most 0.002. Such space is the basis for both token-level explanations (LIME, SHAP)

and case retrieval (FAISS) as well as for the confident decisions made by the agent. It is more reasonable to give explanations of one interpretable model within the same semantic space than to approximate the explanation of the complex meta-learner (Molnar, 2022). Two different explanation techniques were used for every prediction: LIME (Ribeiro et al., 2016) randomly perturbs the input and fits the local linear surrogate, attributing the prediction to specific words. SHAP (Lundberg and Lee, 2017) calculates the Shapley value of every token, showing its contribution. The use of both techniques and evaluation of their agreement helps to prevent reliance on only one method. The context for every prediction is provided using FAISS (Johnson, Douze, and Jégou, 2019): the query is embedded using BioBERT and its closest neighbors in labelled records are found, giving real precedent cases for reviewers along with their domains and similarity scores.

3.8 Agentic triage layer

The triage layer serves as an intelligent decision-making mechanism and not a one-way pipeline (please refer to Figure 3). If the output of the decision process is sure and low in severity, the layer gives an immediate decision without going through the processes of providing an explanation and searching. On the other hand, for those decisions that have uncertainties and higher severity, the layer utilizes LIME/SHAP and FAISS tools for reconsidering the case before directing it to the human reviewer. Cases pass through different computational processes, which are proven by the tool-call counters.

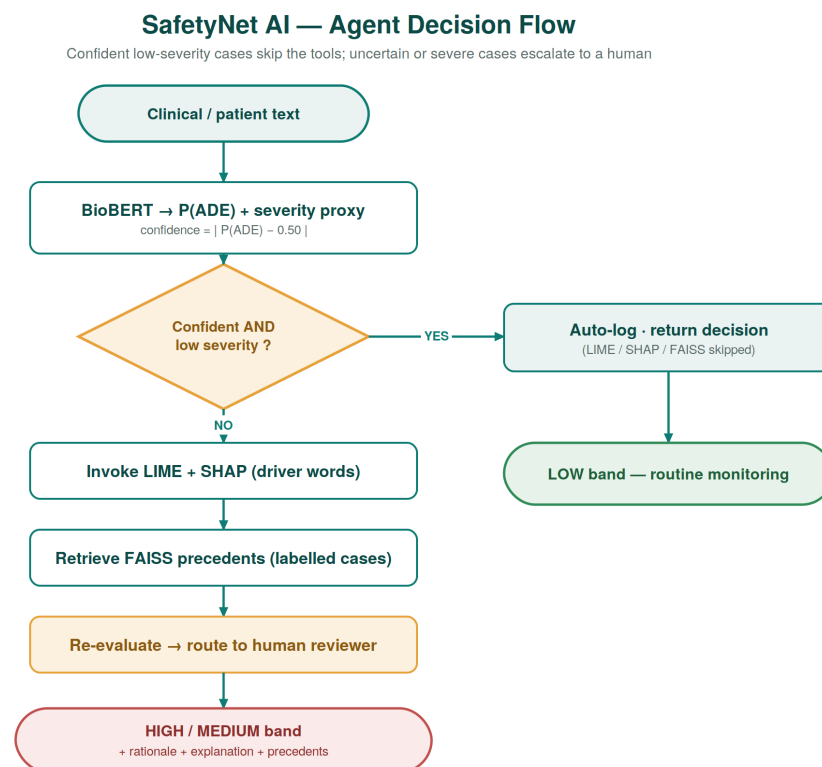


Figure 3. Agent decision flow:

3.9 Cross-domain protocol

Directly measuring generalization, each model is trained only on the formal DailyMed partition and tested only on the informal CADEC partition. This degradation, measured as the difference between the F1 score of same-domain performance and cross-domain F1 score, is the primary metric to answer the research question. The procedure is identical for all models, giving a comparable metric

of degradation, while Section 4 discusses this result considering the class balance and word overlap characteristics presented in Section 2.

Two configurations of BioBERT are used: a model deployed for predicting, explaining and retrieving (Sections 3.5–3.8), and an additional formal-only version only used for measuring degradation according to this cross-domain protocol. The explainability results in Section 4.4 concern the former configuration, not the latter.

3.10 Application demonstration

This system is made up of models and explanation layers that exist within an interactive application. More precisely, it is deployed through a multiple-page Streamlit application together with a Flask web application, enhanced by a FastAPI app to ensure robust deployment. Everything starts with a privacy-consent gate (no personal data, decision-support only, legal processing per UK GDPR/DPA 2018, approved use), ensuring role separation of the submitter and the clinical reviewer and audit logging into an SQLite database. The outputted prediction contains the triage confidence score, driver terms using LIME/SHAP, and FAISS precedents. The corresponding Model Card and Governance page explain the intended use, training datasets, and limitations. With the provided weights of trained BioBERT, the application performs the model; otherwise, it provides a clear lexical preview of the interface. Section 4.5 gives a discussion of a concrete example case.

3.11 Testing, reproducibility and CI/CD

The source code contains an automatically run suite of fourteen pytest test cases, which cover metric functions, probability merging and column-order guard of the ensemble, the retrieval pipeline, inference adapters, and routing of the agent. Continuous integration is enabled with GitHub Actions, which run the tests on each push and control the branching process, meaning that only pull requests can merge feature branches to main. The coding standard is PEP 8, and formatting and linting are done using Black and Flake8, respectively. Pull requests are reviewed by the Code Review Manager, while production release requires consent from the Test Plan Manager and the Team Leader. To guarantee deterministic reproducibility of results, a fixed random seed (42), list of dependencies and provenance (see Appendix B) are kept.

3.12 Hyperparameter optimisation and training diagnostics

The hyperparameter selection process conventionally took place, given the restrictions of computing power available for this project. In particular, no grid search or random search was performed, and the deficiency is addressed in Section 4.8. The precise numbers used can be found in Table 6 for reproducibility of the results obtained from the repository. Those marked as defaults are those that are used by default in the library at the version indicated in requirements.txt. The reasoning behind the choice is provided when needed.

4. Results and Analysis

4.1 Same-domain model comparison

Table 3 presents every model on the held-out same-domain test set (451 records, 60.1% positive). All models achieve more than 0.97 F1 and 0.99 ROC-AUC. However, the headline ranking carries a lesson that a simple reading would miss.

Model	Accuracy	Precision	Recall	F1	Macro-F1	ROC-AUC
TF-IDF + LogReg	0.9823	0.9747	0.9963	0.9854	0.9814	0.9954
SVM-BERT	0.9823	0.9852	0.9852	0.9852	0.9815	0.9921
Ensemble (LR stack)	0.9823	0.9852	0.9852	0.9852	0.9815	0.9982
BioBERT	0.9800	0.9816	0.9852	0.9834	0.9792	0.9949
Bi-LSTM + attention	0.9778	0.9780	0.9852	0.9816	0.9768	0.9968
Random Forest	0.9712	0.9574	0.9963	0.9765	0.9696	0.9948
XGBoost	0.9645	0.9636	0.9779	0.9707	0.9629	0.9918

Table 3. Same-domain test performance, sorted by F1 (source: metrics_*.csv and the ensemble comparison). Highlighted: the best-F1 baseline and the best-ranking ensemble.

The primary conclusion drawn from the findings is that the simplest model has the best F1 score. In other words, the TF-IDF logistic regression gets an F1 of 0.9854, slightly outperforming SVM-BERT and Ensemble, which have F1 scores of 0.9852 each, while BioBERT's F1 score equals 0.9834. The computationally more complex models do not perform better than a simple linear model based on word counts in the same-domain F1 evaluation. Combining this information with exploratory data analysis, the conclusion appears to be reasonable since the ADE signal in this dataset is mostly determined by word selection. Distinct words like "adverse," "reaction," and "warning" are successfully separating the classes so that a simple bag-of-words linear model can exploit almost all the signal with very little benefit from context comprehension.

It becomes clear that advanced models outperform others in ranking quality rather than in F1 with a certain threshold. The stacking ensemble achieves the highest ROC-AUC value – 0.9982, which is higher than any other model, with Bi-LSTM just behind with ROC-AUC 0.9968. ROC-AUC measures the quality of ranking cases by risk regardless of the 0.5 decision threshold. Such a property is crucial for a triage system because it provides an opportunity for making decisions based on the level of confidence and elevating borderline cases. Hence, while having equal hard-label F1 with SVM-BERT, the ensemble gives probabilities that are more calibrated and ranked. Such discrepancy between classification on one threshold and ranking on all thresholds is very important for interpreting Table 3 correctly.

Finally, there is a small artifact related to the number of test samples: the recall across all robust models is 0.9852. Having 271 positive samples in the test set, this means that they detect 267 cases but miss four borderline samples. The equality arises from the limited number of held-out samples and does not mean any agreement between models. Still, these in-domain results are just half of the truth; as will be shown in Section 4.2, this equality disappears in a cross-domain setting.

4.2 Cross-domain generalisation: The core result

Each model was trained from scratch on one domain and evaluated on both, following the protocol in Section 3.9. Splitting is by content hash rather than by row, so every copy of a repeated text is assigned to the same side of the split; an automatic guard (assert_no_leakage) then hashes each record and verifies that all four train/test pairs are disjoint, halting the experiment if any record appears in both. Under this protocol, the formal test partition contains 456 records (39.7% ADE-

positive) and the informal test partition 312 records (91.0% ADE-positive). Table 4 reports the full grid on macro-F1; Table 4a repeats it on binary F1, and Table 4b gives the supporting diagnostics.

Model	F→F	I→I	F→I	I→F
TF-IDF + LogReg	0.9977	0.7629	0.0824	0.2963
TF-IDF + RF	1.0000	0.7418	0.0824	0.6252
TF-IDF + XGB	1.0000	0.7151	0.0898	0.7500
Bi-LSTM + attention	1.0000	0.7692	0.0824	0.3965
SVM-BERT	0.9931	0.8044	0.1089	0.2859
BioBERT	1.0000	0.8745	0.1017	0.3620

Table 4. Cross-domain transfer grid (macro-F1). Left of the divide is in-domain ($F \rightarrow F$, $I \rightarrow I$); right is cross-domain ($F \rightarrow I$, $I \rightarrow F$). The Formal $\rightarrow$ Informal column (highlighted) is the deployment-critical test. Macro-F1 is reported because binary F1 collapses to zero under degenerate majority-class prediction and cannot distinguish degrees of

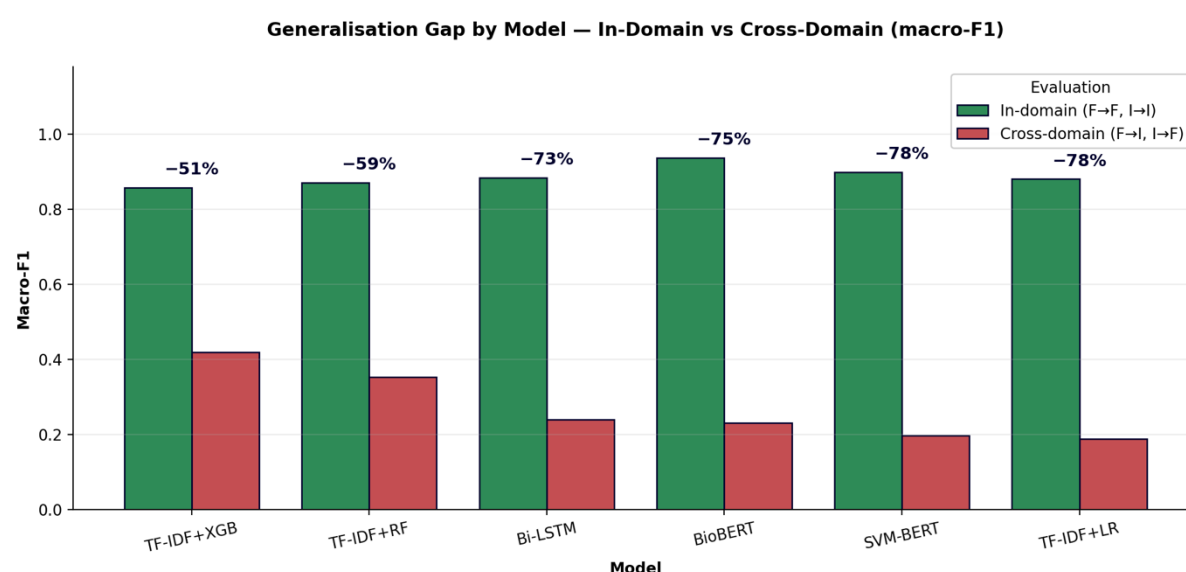


Figure 4. Generalisation gap by model: mean in-domain macro-F1 ($F \rightarrow F$, $I \rightarrow I$) against mean cross-domain macro-F1 ($F \rightarrow I$, $I \rightarrow F$), annotated with the relative drop. Derived from Table 4

Three findings stand out.

Finding 1: the Formal to Informal transfer fails for every model.

All six models score between 0.0824 and 0.1089 macro-F1 on this transfer, with balanced accuracy between 0.493 and 0.504 against a chance level of 0.500. The binary F1 figures are starker still: three of the six logistic regression, random forest, and the Bi-LSTM models score exactly 0.000, predicting no positive case at all on a target partition that is 91.0% ADE-positive. Across the six models, the predicted positive rate ranges from 0.0% to 2.9% against that 91.0% prevalence (Table 4b). No model trained on regulatory product labelling retains any usable ability to identify adverse events in patient narrative. This is not a degradation; it is a complete loss of the decision function.

Finding 2: biomedical pre-training conferred no transfer advantage whatsoever.

BioBERT is the strongest in-domain model in the study, with the highest mean in-domain macro-F1 (0.9373) and the best result on the harder informal partition (0.8745, against 0.715–0.804 for the other five). It also records the second-largest generalisation gap of the six, at 75.3%. More telling than its rank is the shape of its failure: BioBERT is the only model whose Formal-to-Informal ROC-AUC falls below chance, at 0.492, meaning its formal-domain representation is not merely

uninformative on patient text but slightly anti-correlated with the target. Every other model retains at least weakly positive ranking ability on that transfer (0.544–0.664). The most plausible reading is that fine-tuning on regulatory labelling taught the model structural regularities of that genre (section headings, list formatting, formulaic phrasing) which are systematically absent from, or inverted in, patient narrative. This contradicts the expectation implied by the biomedical pre-training literature (Lee et al., 2020; Gururangan et al., 2020) and is the clearest negative result of the project.

Finding 3: The best transfer belongs to the models that resist inheriting their source prior.

The strongest cross-domain model is TF-IDF with XGBoost (mean cross-domain macro-F1 0.4199, degradation 51.0%), followed by TF-IDF with random forest (0.3538, 59.4%). Both are lexical baselines, and both carry explicit inverse-frequency class weighting. The advantage is not an artefact of class imbalance but a consequence of resisting it: on Informal to Formal, XGBoost predicts ADE for 61.6% of records against a true prevalence of 39.7%, and random forest for 75.4%, while the remaining four models predict between 93.4% and 99.3%. XGBoost is the only model to recall more than 60% of the negative class on that transfer (0.611, against 0.011 for logistic regression). The pattern is consistent across the tier: the models that transfer best are those least willing to reproduce the class prior of the register they were trained on. This should be read cautiously. The result rests on a single seed, one register pair, and a baseline tier in which imbalance handling was deliberately equalised (Section 3.2), so XGBoost’s advantage is partly a consequence of the explicit prior correction built into its training objective, which is itself the mechanism this section identifies.

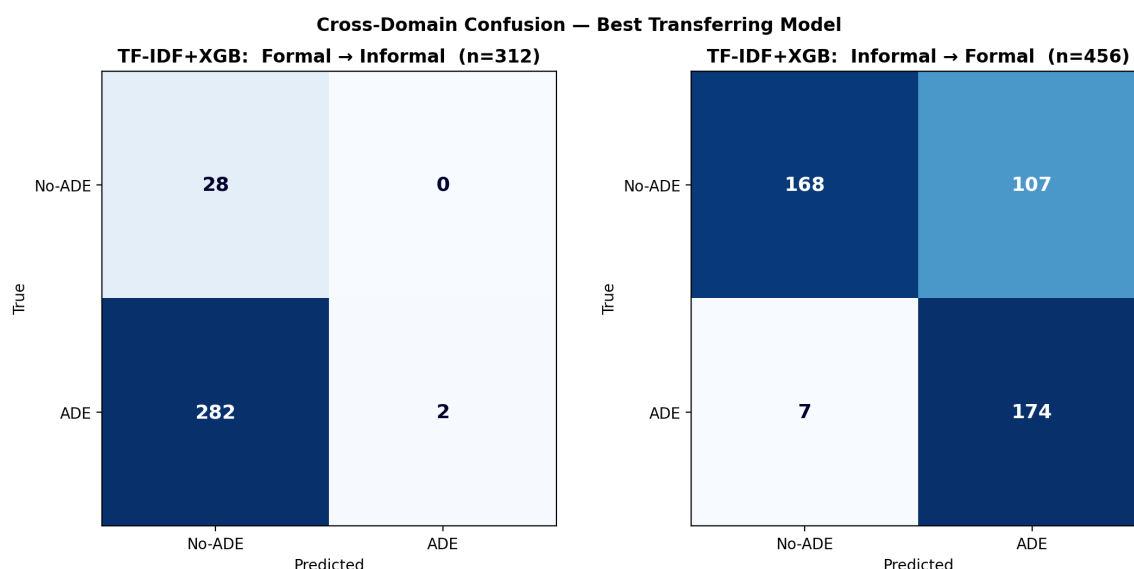


Figure 5. TF-IDF + XGBoost cross-domain confusion matrices, the best-transferring model

Table 5 sets the results against the "excellent" tier of the project specification. Every in-domain F1 target was exceeded by a wide margin. The cross-domain degradation target of $\leq 10\%$ was not met by any model: the best transfer, TF-IDF with XGBoost, degrades by 51.0%, and the remaining five by between 59.4% and 78.5%. We report this as a finding rather than a shortfall. A $\leq 10\%$ target presumes that formal regulatory labelling and patient narrative are close enough for a single decision function to span both; the top-300 vocabulary Jaccard of 0.143 (Section 2.3) and the prior shift documented above show that they are not. The value of the result lies in measuring and explaining the gap, which this section and Section 4.3 do.

Target (specification)	“Excellent” goal	Achieved	Met?
Best baseline F1 (in-domain)	≥ 80%	98.5%	Yes
Bi-LSTM + attention F1 (in-domain)	≥ 87%	98.2%	Yes
SVM–BERT F1 (in-domain)	≥ 89%	98.5%	Yes
BioBERT F1 (in-domain)	≥ 90%	98.3%	Yes
Ensemble F1 (in-domain)	≥ 92%	98.5%	Yes
Cross-domain degradation	≤ 10%	51.0% best (TF-IDF + XGB); 59.4–78.5% others	No : reported as the central finding

Table 5. Outcomes against the project specification's targets. In-domain F1 values are from the pooled same-domain test set (Table 3)

4.2.1 Prior correction: separating threshold shift from representational failure

If the collapse in Table 4 were a wholesale representational failure, ranking quality would collapse with it, and ROC-AUC would sit at chance. If instead the models are inheriting the class prior of their training register, ranking should survive, and only the decision threshold should be wrong. The two are separable without any target labels: setting the threshold to the quantile of the predicted probabilities that makes the predicted positive rate equal the target's known prevalence corrects the prior while leaving the ranking untouched. Class prevalence is known for both registers (Table 1), so the correction requires no labelled target data.

The results in Table 4b (appendix) show that the two directions fail for different reasons. On Informal→Formal, ranking largely survives. The Bi-LSTM achieves an ROC-AUC of 0.995, the highest of any transfer in the study, while scoring only 0.3965 macro-F1 at the default threshold, because it predicts ADE for 93.4% of a target that is 39.7% positive. Prior correction recovers it to 0.9908 macro-F1. Logistic regression shows the same pattern less dramatically, moving from 0.2963 to 0.8214 at a ROC-AUC of 0.922. Across all six models, the mean Informal→Formal macro-F1 rises from 0.453 to 0.747. Almost the entire deficit in this direction was threshold placement, not representation.

On Formal→Informal, correction does not rescue the transfer. The mean rises from 0.091 to 0.512, but no model exceeds 0.5684, because ranking has itself collapsed: ROC-AUC on this transfer ranges from 0.492 to 0.664. There is no threshold that recovers a model which cannot rank the target. The apparent improvement is largely an artefact of the target's 91.0% positive prevalence; a corrected model predicting 91% positives scores high binary F1 almost by construction, which is precisely why macro-F1 is the metric reported here.

The distinction matters for practice. Where a detector is moved to a register whose prevalence is known and whose language it can still rank, prior correction is a cheap and label-free remedy. Where the register shift has destroyed ranking, as it has from regulatory labelling to patient narrative, no threshold adjustment substitutes for training data in the target register.

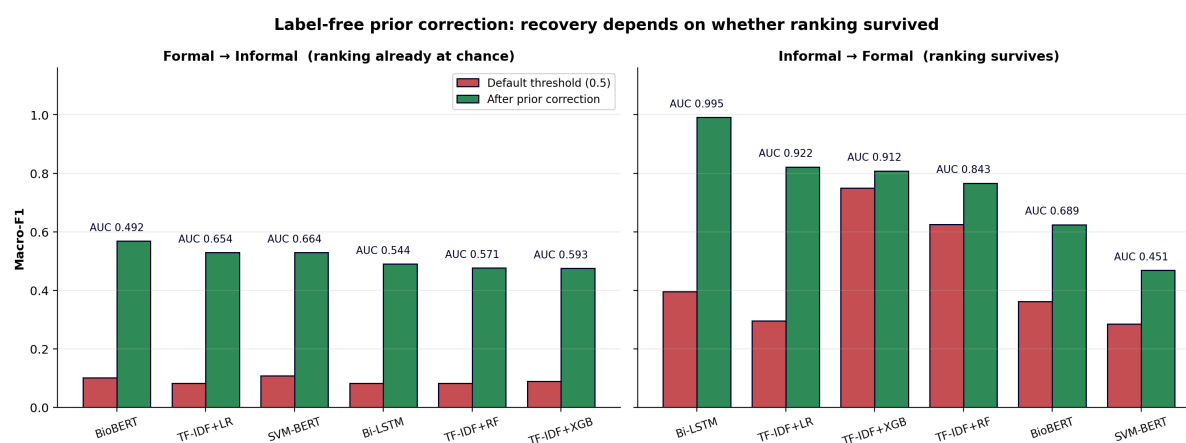


Figure 9. Label-free prior correction on both transfer directions. Recovery depends on whether ranking survived: Informal→Formal recovers substantially, Formal→Informal does not. ROC-AUC is annotated above each pair.

4.3 Comparison with the literature

Our in-domain performance sits at the upper end of the published range. Huynh et al. (2016) report neural architectures, convolutional and recurrent, reaching F1 between 0.80 and 0.90 on ADR benchmark corpora, and Lee et al. (2020) report transformer results in a comparable band; every model in Table 3 exceeds 0.97 binary F1. The gap reflects the task definition rather than any superiority of our models. We perform record-level binary classification, whereas the benchmark work performs token-level span identification, a substantially harder objective; and our harmonised corpus is more lexically separable than those benchmarks, since the formal partition derives its labels from LOINC-coded label sections whose vocabulary is close to deterministic (Section 2.1). The comparison is therefore not like-for-like, and the in-domain figures should be read as evidence about the corpus rather than about the models. That the TF-IDF baseline edges out fine-tuned BioBERT in-domain (0.9854 against 0.9834 binary F1) illustrates a familiar pattern: classical baselines remain competitive when the signal is carried by lexical choice, and the benefit of contextual models is realised only where disambiguation is required. That condition is met on the informal partition, where BioBERT reaches 0.8745 macro-F1 against 0.715–0.804 for the other five models (Table 4, I→I column), the one setting in which biomedical pre-training pays for itself. Note the metric change: the in-domain comparison is binary F1 on the pooled test set, while the informal-partition comparison is macro-F1 on the per-domain split, so the two figures are not directly comparable to one another.

Our results also sharpen the existing literature on cross-domain performance. Sarker and Gonzalez (2015) and Weissenbacher et al. (2019) report limited transferability across genres; under our leakage-free protocol, the effect is more severe still, with every model scoring between 0.0824 and 0.1089 macro-F1 on Formal to Informal and balanced accuracy at chance. The caveat concerns pre-training. Lee et al. (2020) and Gururangan et al. (2020) both advocate pre-training on biomedical material, the latter showing that in-domain pre-training yields gains proportional to domain distance, but neither considers cross-register transfer of the kind tested here, and in our setting the expected order is close to reversed. The strongest cross-domain models are two lexical TF-IDF baselines (XGBoost at 0.4199 mean macro-F1, random forest at 0.3538), while BioBERT, the strongest in-domain model at 0.9373 degrades by 75.3% and is the only model whose Formal-to-Informal ranking falls below chance, at 0.492 ROC-AUC. Fine-tuning on regulatory labelling appears to have transferred genre knowledge rather than the phenomenon itself.

Our contribution is the mechanism. Ben-David et al. (2010) bound target-domain error by inter-domain divergence, and our top-300 vocabulary Jaccard of 0.143 measures that divergence directly. But divergence alone does not explain why some transfers fail at the default threshold while continuing to rank well, and others do not. On Informal to Formal, the Bi-LSTM reaches an ROC-AUC of 0.995 while scoring only 0.3965 macro-F1, and a label-free prior correction recovers it to 0.9908; logistic regression follows the same pattern at ROC-AUC 0.922, moving from 0.2963 to 0.8214. On Formal to Informal, no such recovery is available, because ROC-AUC there ranges only from 0.492 to 0.664; the ranking signal is gone, and correction lifts the six-model mean from 0.091 to just 0.512 (Section 4.2.1). The implication for practice is that reporting positive-class F1 alone conceals two distinguishable conditions, recoverable threshold shift and irrecoverable representational failure, and that separating them requires macro-F1, balanced accuracy and ROC-AUC read together.

4.4 Explainability results

The LIME vs. SHAP features at the token level for some exemplary samples are illustrated in Figure 6. As one can see from the LIME table, it mainly uses medical terminology such as “adverse,” “seizure,” “during,” and “volunteers.” In its turn, the SHAP table focuses on informal elements like “pain,” “bad,” “grouchy,” and “stop.” These facts illustrate that the classifier works with terms specific to each register, which confirms the claims discussed in Section 4.2.

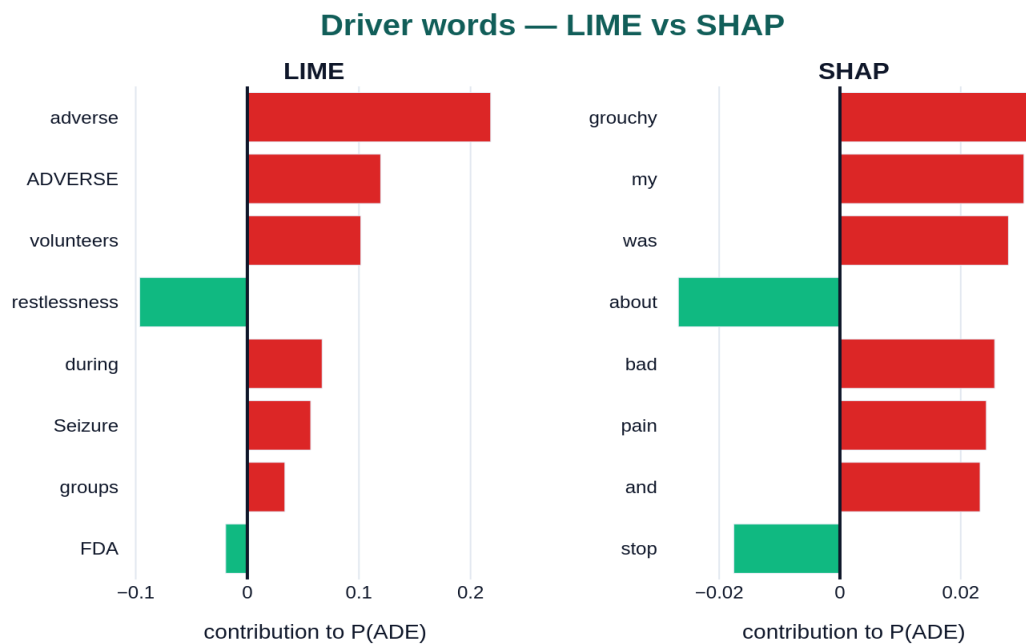


Figure 6. Token-level ADE drivers from LIME and SHAP (own work). Red = pushes towards ADE; green = pushes away.

These examples point to a substantial weakness as well. Certain high-frequency tokens are non-clinical in nature; these include “my,” “was,” and “and.” The frequency suggests that the algorithm uses register, i.e., how patients write their queries, not just the clinical information embedded there. This example is important for reviewers of decision support systems because it highlights the importance of human involvement and precedent retrieval in the system as a mandatory process.

The FAISS retrieval layer is demonstrated in Figure 7 below. For a particular query, the layer finds its nearest neighbors in the BioBERT embedding space, alongside the similarities, labels, and domains of the neighbor precedents. In the example provided, all ten nearest neighbors are formal, ADE-positive precedents, with similarity around 0.81. Thus, the reviewer has a labeled precedent that

may either support or disprove the prediction made by the algorithm.

Similar known cases — FAISS (BioBERT space)

#	Similarity	Label	Domain	Case (retrieved precedent)
1	0.816	ADE	Formal (DailyMed)	adverse reaction following discussed detail section labeling increased mortality elderly patient dementiarelat...
2	0.816	ADE	Formal (DailyMed)	adverse reaction following adverse reaction described greater detail section acute lithium toxicity see warnin...
3	0.816	ADE	Formal (DailyMed)	adverse reaction following adverse reaction described greater detail section acute lithium toxicity see warnin...
4	0.816	ADE	Formal (DailyMed)	adverse reaction following discussed detail section labeling known hypersensitivity methylphenidate product in...
5	0.814	ADE	Formal (DailyMed)	adverse reaction following adverse reaction discussed greater detail section labeling known hypersensitivity a...
6	0.814	ADE	Formal (DailyMed)	adverse reaction following adverse reaction discussed greater detail section labeling known hypersensitivity a...

10/10 nearest precedents are ADE-positive — supporting evidence for the model's call

Figure 7. FAISS retrieval of similar precedent cases in BioBERT embedding.

As mentioned in section 3.7, these rationales are relevant to BioBERT in particular rather than to the ensemble model. The outstanding performance of the model as well as the logical structure of the embedding space make it a relevant candidate for further examination. In addition, the attention layer of the Bi-LSTM offers information on the tokens that were considered important by the model.

4.5 Application demonstration analysis

A typical example from the triage interface is shown in Figure 8, where the classifier provides a probability for P(ADE) of 99.4%, placing the case into the “High, urgent review” category, which is 49.4 points higher than the decision threshold of 0.5, along with moderate severity. With both high confidence and severity associated with the prediction, the agent uses explanation and retrieval modules (see Figures 6–7) to send the case for human evaluation rather than automatic filtering. This is consistent with the design of the system, where values with a high degree of confidence far away from the decision threshold are disclosed, while the severity flag triggers the human-in-the-loop mechanism

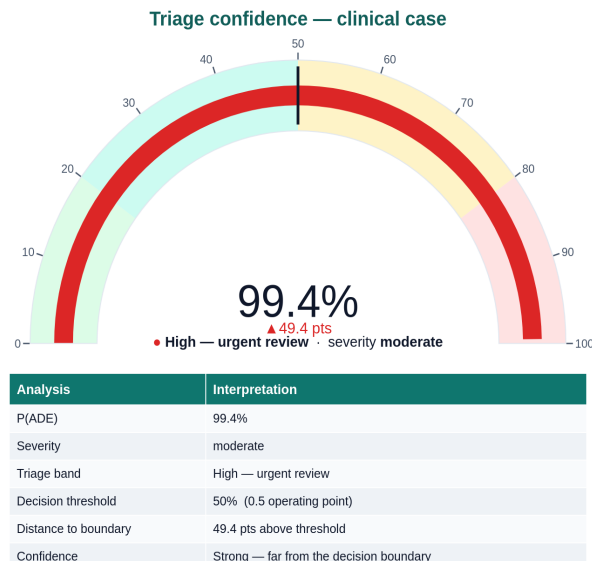


Figure 8. Triage confidence reading for a clinical case in the SafetyNet AI application.

The demo tests the system fully, as the raw text entry is predicted, routed for severity, explained twice, and preceded in order to reach the triage decision through the governance gates and audit logging. Additionally, the demo makes clear the limitations of the project, as the severity proxy is marked as derived, and the confidence score as decision-support information.

4.6 Answering the research question

Returning to the research question: can a classifier trained on formal drug-label text detect adverse drug events in informal patient text, and can an explainable, agentic layer make such a system clinically usable? The empirical answer is no; the system's answer is yes. Transfer fails uniformly. Every model, including the biomedically pre-trained BioBERT, scores between 0.0824 and 0.1089 macro-F1 on the Formal→Informal transfer, with balanced accuracy at chance, and three of the six predict no positive case at all. Pre-training on biomedical corpora conferred no transfer advantage: BioBERT records the only below-chance Formal→Informal ROC-AUC of the six, at 0.492. The mechanism is registering divergence: the two vocabularies share a top 300 Jaccard similarity of 0.143 (Section 2.3) compounded by inherited class priors, analysed in Sections 4.2 and 4.2.1. A detector trained on one register therefore cannot be deployed on the other without explicit domain adaptation, target-register training data, or, where ranking survives, prior correction. On the systems question the evidence is positive: LIME/SHAP attribution, FAISS precedent retrieval and the agentic human-in-the-loop layer make each prediction inspectable and safely routable, and the deployed model is trained on the pooled corpus and is therefore in-domain for both registers by construction, so it is not subject to the failure documented here.

4.7 Applications and use-cases

The evidence in this chapter supports several concrete applications, each with a different tolerance for the failure modes identified above. Regulatory signal triage. National pharmacovigilance schemes receive far more free-text reports than can be manually reviewed, and a detector with reliable ranking is well suited to ordering an incoming queue by risk. The ensemble is the appropriate model here, because ROC-AUC, the metric on which it leads, measures exactly the ranking quality the task requires. The caveat is that spontaneous-report narrative is itself a third register, closer to CADEC

than to DailyMed, so a model deployed there should be trained on report text rather than on product labelling. Patient-narrative surveillance. This is the use case that motivated the cross-domain experiment and the one the results most restrict. No model transfers into it from regulatory labelling, and biomedical pre-training does not rescue the transfer. The recommendation is therefore not a choice of encoder but a change of protocol: train on the target register or on a pooled corpus spanning both, as the deployed system does, or where target labels are unavailable, and ranking survives, apply prior correction (Section 4.2.1). Hospital pharmacy and clinical review support. Free-text nursing notes and discharge summaries contain adverse-event evidence that is rarely coded systematically. The explainability and retrieval layers matter most here, and the agentic routing layer is what makes such a system usable at ward volume without removing human judgement; note that clinical notes are a fourth register on which no model in this study was trained. Pharmaceutical post-market surveillance. Marketing-authorisation holders must monitor safety signals across heterogeneous sources, and the transfer matrix in Table 4 is directly useful to such an organisation as a decision aid: it states which model families can and cannot be reused across registers, information usually discovered only after deployment failure. Preconditions for deployment. None of these applications is reachable from the current prototype without further work. The severity signal must be replaced by a clinically validated label before it can drive routing, since it is currently a derived heuristic. The system requires prospective validation on live data rather than a retrospective held-out split. Under UK regulation, software used to inform clinical decisions would fall within the medical-device software framework and require the corresponding conformity assessment. Finally, the near-perfect in-domain scores in Section 4.1 reflect a lexically separable corpus and should not be presented to a prospective user as an expectation of field performance. The system is positioned throughout as decision support with a human in the loop, and that positioning is a safety requirement rather than a disclaimer.

4.8 Limitations

Several limitations bound the results above. Hyperparameters were not tuned by grid or random search, for the compute reasons given in Section 3.12; the values in Table 6 are library defaults or conventional settings, so the model comparison should be read as a comparison of families rather than of optimally configured instances. All cross-domain results rest on a single random seed, one register pair and one direction of interest, so the ranking of models within the cross-domain column is not established with confidence.

Three protocol details qualify the comparison. First, the Bi-LSTM and SVM-BERT tiers previously monitored training on their own training split; both now hold out a 15% stratified internal validation slice, matching BioBERT, but earlier reported values for those tiers are not comparable to the present ones. Second, all six models in the cross-domain experiment consume the lightly normalised text_processed field rather than raw text, including the cased BioBERT checkpoint; this was done for comparability across tiers, but the casing mismatch may contribute to BioBERT's poor transfer and has not been isolated. Third, XGBoost carries an explicit per-domain class weighting introduced so that imbalance handling does not differ across the baseline tier (Section 3.2); since class-prior shift is the mechanism this chapter identifies, XGBoost's strong transfer is partly a consequence of that design choice rather than an independent finding.

An earlier iteration of the cross-domain experiment substituted the deployed BioBERT checkpoint, fine-tuned on the pooled training split, for a formal-only model. Because roughly 70% of informal records sit in that pooled split, the Formal→Informal cell was scoring the model against records it had already seen, producing a spuriously high transfer score that contradicted every other model in the matrix. The discrepancy was traced to the model cache path, and the protocol was corrected to fine-tune BioBERT separately per domain with the domain name embedded in the cache path. Separately, the original per-domain splitter divided the data by row rather than by content hash, allowing repeated texts to appear on both sides of a split; 45 formal records were affected. Both

defects were found and fixed before the results reported here were produced, and the leakage guard now runs against the split object actually used.

Finally, the severity signal used for triage routing is a derived heuristic rather than a clinically validated label, and the near-perfect formal in-domain scores reflect the near-deterministic nature of LOINC-derived section labelling rather than model capability. Neither should be read as an expectation of field performance.

5. Conclusions and Future Work

5.1 Conclusions

This project was designed to detect ADEs in formal and informal textual data sources in a way that is both explainable and actionable for clinicians. This project yielded a cross-domain corpus of 3,077 records, a set of models from simple TF-IDF baselines to a stacked ensemble measured against one metric formulation, and an explainable agentic triage app with an overt approach to governance. All five objectives were addressed. The in-domain targets were exceeded, and the cross-domain objective was met as an analytical objective; the generalisation gap was measured under a verified leakage-free protocol and its mechanism explained, although the $\leq 10\%$ degradation target itself was not achieved. The value of the research is mostly in the thoroughness of its analysis rather than any single accuracy measure.

Three main conclusions can be drawn. First, ADE detection within this corpus is mostly a lexical problem: a simple TF-IDF logistic-regression model ($F1 = 0.9854$) rivals or beats fine-tuned BioBERT, suggesting that the additional complexity of transformer architecture brings little gain in same-domain classification. Second, the more complex models shine in other tasks: stacked ensemble yields optimal risk-ranking ($ROC-AUC = 0.9982$), and BioBERT embedding space is used in retrieval and explanation layers. And third, and most importantly, the leakage-free cross-domain test showed a large and uniform generalisation gap. No model transfers from formal labelling to patient narrative: all six score between 0.0824 and 0.1089 macro-F1 with balanced accuracy at chance, three of them predicting no positive case at all, and degradation ranges from 51.0% for the best model to 78.5% for the worst. Biomedical pre-training did not help; BioBERT is the strongest in-domain model and the only one whose Formal→Informal ranking falls below chance. The failure is directional and largely attributable to inherited class priors: where ranking survives, as on Informal→Formal, a label-free prior correction recovers most of the loss (the Bi-LSTM from 0.397 to 0.991 macro-F1); where ranking has collapsed, as on Formal→Informal, no threshold adjustment substitutes for target-register training data. Together with the explainability, retrieval and human-in-the-loop triage layers, these results make SafetyNet AI both a functional prototype and a demonstration of why headline F1 on imbalanced clinical corpora requires scrutiny.

5.2 Future work

- Consider using balanced accuracy, macro F1, and per-class recall as primary metrics for evaluating the cross-domain performance, and balance/re-sample the informal dataset so that the transfer is justified not through the F1 scores.
- Replace the severity proxy with a medically validated severity label, even better if supplemented with the physician's annotation, as the triage should be done based on the true severity score and not the proxy.

- Extend the dataset with more informal and formal documents for testing transfer between register pairs and expanding the small negative class of informal examples.
- Move from record-level classification to token-level span extraction, which is a more difficult and medically meaningful task, allowing the explanation module to extract the exact adverse reaction spans.
- Perform more calibration and analysis of the ensemble to make sure that the confidence values used for triage decisions are as reliable as the ranking metric claims.
- Conduct a small human study with a reviewer trying to improve the triage based on explanations and precedents provided.

References

- Bahdanau, D., Cho, K. and Bengio, Y. (2015) 'Neural machine translation by jointly learning to align and translate', 3rd International Conference on Learning Representations (ICLR 2015), San Diego, 7–9 May. Available at: <https://arxiv.org/abs/1409.0473> (Accessed: 6 July 2026).
- Ben-David, S., Blitzer, J., Crammer, K., Kulesza, A., Pereira, F. and Vaughan, J.W. (2010) 'A theory of learning from different domains', *Machine Learning*, 79(1–2), pp. 151–175.
- Chen, T. and Guestrin, C. (2016) 'XGBoost: a scalable tree boosting system', *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. San Francisco: ACM, pp. 785–794.
- Devlin, J., Chang, M.-W., Lee, K. and Toutanova, K. (2019) 'BERT: pre-training of deep bidirectional transformers for language understanding', *Proceedings of NAACL-HLT 2019*. Minneapolis: Association for Computational Linguistics, pp. 4171–4186.
- Data Protection Act 2018, c. 12. Available at: <https://www.legislation.gov.uk/ukpga/2018/12/contents> (Accessed: 14th July 2026).
- Gurulingappa, H., Rajput, A.M., Roberts, A., Fluck, J., Hofmann-Apitius, M. and Toldo, L. (2012) 'Development of a benchmark corpus to support the automatic extraction of drug-related adverse effects from medical case reports', *Journal of Biomedical Informatics*, 45(5), pp. 885–892.
- Gururangan, S., Marasović, A., Swayamdipta, S., Lo, K., Beltagy, I., Downey, D. and Smith, N.A. (2020) 'Don't stop pretraining: adapt language models to domains and tasks', *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, pp. 8342–8360.
- Hochreiter, S. and Schmidhuber, J. (1997) 'Long short-term memory', *Neural Computation*, 9(8), pp. 1735–1780.
- Huynh, T., He, Y., Willis, A. and Rüger, S. (2016) 'Adverse drug reaction classification with deep neural networks', *Proceedings of COLING 2016*. Osaka: The COLING 2016 Organizing Committee, pp. 877–887.
- Johnson, J., Douze, M. and Jégou, H. (2019) 'Billion-scale similarity search with GPUs', *IEEE Transactions on Big Data*, 7(3), pp. 535–547.
- Karimi, S., Metke-Jimenez, A., Kemp, M. and Wang, C. (2015) 'CADEC: a corpus of adverse drug event annotations', *Journal of Biomedical Informatics*, 55, pp. 73–81.
- Lee, J., Yoon, W., Kim, S., Kim, D., Kim, S., So, C.H. and Kang, J. (2020) 'BioBERT: a pre-trained biomedical language representation model for biomedical text mining', *Bioinformatics*, 36(4), pp. 1234–1240.
- Lundberg, S.M. and Lee, S.-I. (2017) 'A unified approach to interpreting model predictions', *Advances in Neural Information Processing Systems 30*. Long Beach, CA: Curran Associates, pp. 4765–4774.
- Molnar, C. (2025) *Interpretable machine learning: a Guide for Making black box Models Explainable*. 3rd Edition. Available at: christophm.github.io/interpretable-ml-book/ (Accessed: 5th July 2026).
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, É. (2011) 'Scikit-learn: machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830.

- Ribeiro, M.T., Singh, S. and Guestrin, C. (2016) “‘Why should I trust you?’: explaining the predictions of any classifier”, Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. San Francisco: ACM, pp. 1135–1144.
- Sarker, A. and Gonzalez, G. (2015) ‘Portable automatic text classification for adverse drug reaction detection via multi-corpus training’, Journal of Biomedical Informatics, 53, pp. 196–207.
- U.S. National Library of Medicine (2026) DailyMed. Available at: <https://dailymed.nlm.nih.gov/dailymed/> (Accessed: 16 June 2026).
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł. and Polosukhin, I. (2017) ‘Attention is all you need’, Advances in Neural Information Processing Systems 30. Long Beach, CA: Curran Associates, pp. 5998–6008.
- Weissenbacher, D., Sarker, A., Magge, A., Daughton, A., O’Connor, K., Paul, M.J. and Gonzalez-Hernandez, G. (2019) ‘Overview of the fourth Social Media Mining for Health (SMM4H) shared tasks’, Proceedings of the Fourth SMM4H Workshop. Florence: Association for Computational Linguistics, pp. 21–30.
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M. and Brew, J. (2020) ‘Transformers: state-of-the-art natural language processing’, Proceedings of EMNLP 2020: System Demonstrations. Association for Computational Linguistics, pp. 38–45.
- Wolpert, D.H. (1992) ‘Stacked generalization’, Neural Networks, 5(2), pp. 241–259.

Appendices

Appendix A — Repository structure and code

Code for Preprocessing and EDA:

```
!pip install -q pandas numpy matplotlib seaborn nltk scikit-learn
requests

import os
import re
import glob
import time
import json
import logging
from datetime import datetime, timezone
from typing import Any, Dict, List, Optional, Tuple

import requests
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry
import pandas as pd
import numpy as np
import xml.etree.ElementTree as ET
from collections import Counter
import matplotlib.pyplot as plt
import seaborn as sns
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from sklearn.model_selection import GroupShuffleSplit
import warnings
warnings.filterwarnings("ignore")

# levelled logging instead of bare print()
logging.basicConfig(level=logging.INFO, format="[% (levelname)s]
%(message)s")
logger = logging.getLogger("ade_pipeline")

nltk.download("stopwords", quiet=True)
nltk.download("wordnet", quiet=True)
nltk.download("omw-1.4", quiet=True)
sns.set_style("whitegrid")
plt.rcParams["figure.figsize"] = (12, 6)

# Project-wide config
N_DAILYMED = 1500
```

```

RANDOM_SEED = 42
NS = "{urn:hl7-org:v3}"
LOINC_ADVERSE = "34084-4"      # ADVERSE REACTIONS    -> positive (label
1)
LOINC_INDICATIONS = "34067-9" # INDICATIONS & USAGE -> real negative
(0)
LOINC_DOSAGE = "34068-7"      # DOSAGE & ADMIN        -> fallback
negative

# formal schema as a single source of truth + validator
OUTPUT_SCHEMA: Dict[str, Dict[str, Any]] = {
    "text": {"dtype": "object", "desc": "Original raw text"},
    "text_processed": {"dtype": "object",
                        "desc": "Lemmatised, stopword-removed, space-
joined"},
    "label": {"dtype": "int", "desc": "0 = no ADE, 1 = ADE"},
    "severity": {"dtype": "object",
                 "values": ["none", "mild", "moderate", "severe"],
                 "desc": "PROXY ONLY - heuristic, not ground truth"},
    "severity_source": {"dtype": "object", "desc": "Always
heuristic_proxy"},
    "domain": {"dtype": "object",
               "values": ["Formal (DailyMed)", "Informal (CADEC)"]},
    "group_id": {"dtype": "object",
                 "desc": "Leakage key: dm_<setid> or cadec_<file>"},
    "word_count_processed": {"dtype": "int",
                             "desc": "Token count of text_processed"},
}

def validate_schema(df: pd.DataFrame,
                    schema: Dict[str, Dict[str, Any]] = OUTPUT_SCHEMA)
-> None:
    """Assert a DataFrame matches OUTPUT_SCHEMA (Fix #3).

    Checks every required column is present, dtypes are compatible,
    labels
    are binary, and no processed text is empty.
    """
    for col, meta in schema.items():
        assert col in df.columns, f"Missing column: {col}"
        if meta["dtype"] == "int":
            assert pd.api.types.is_integer_dtype(df[col]), \
                f"{col}: expected int, got {df[col].dtype}"
        assert df["label"].isin([0, 1]).all(), "label must be 0/1"
        assert (df["text_processed"].str.len() > 0).all(), \
            "empty text_processed found"
    logger.info("Schema valid: %d rows x %d cols", len(df),
len(schema))

```

```

# HTTP session with retries + exponential backoff
def get_session_with_retries(retries: int = 3,
                             backoff: float = 0.3) -> requests.Session:
    """Return a requests.Session that retries transient API
    failures."""
    session = requests.Session()
    strategy = Retry(
        total=retries,
        backoff_factor=backoff,
        status_forcelist=[429, 500, 502, 503, 504],
        allowed_methods=["GET"],
    )
    session.mount("https://", HTTPAdapter(max_retries=strategy))
    return session

SESSION = get_session_with_retries()
logger.info("Setup ready")

```

Extract dailymed dataset(formal domain):

```

def fetch_drug_index(num_drugs: int) -> List[Dict[str, Any]]:
    """Retrieve metadata for num_drugs labels from the DailyMed API.

    Paginated; retries transient failures via SESSION. Logs and stops
    early
    if the API returns an empty page mid-pull (Issue #1).
    """
    base =
"https://dailymed.nlm.nih.gov/dailymed/services/v2/spls.json"
    drugs: List[Dict[str, Any]] = []
    page = 1
    while len(drugs) < num_drugs:
        r = SESSION.get(base, params={"pagesize": 100, "page": page},
                        timeout=30)
        r.raise_for_status()
        payload = r.json()
        batch = payload.get("data", [])
        if not batch:
            logger.warning("API returned empty page %d - stopping early
"
                           "(requested %d, got %d).", page, num_drugs,
                           len(drugs))
            break
        drugs.extend(batch)
        total_pages = payload.get("metadata", {}).get("total_pages",
page)

```

```

        logger.info("    index page %d/%s (+%d, total %d)", page,
total_pages,
                    len(batch), len(drugs))
        if page >= total_pages:
            break
        page += 1
    return drugs[:num_drugs]

def extract_section_text(root: ET.Element, loinc: str) ->
Optional[str]:
    """Return cleaned text of the XML section with the given LOINC
code."""
    for section in root.iter(f"{NS}section"):
        c = section.find(f"{NS}code")
        if c is not None and c.get("code") == loinc:
            parts: List[str] = []
            for el in section.iter():
                if el.text and el.text.strip():
                    parts.append(el.text.strip())
                if el.tail and el.tail.strip():
                    parts.append(el.tail.strip())
            return re.sub(r"\s+", " ", " ".join(parts)).strip()
    return None

def drug_name(title: str) -> str:
    """Clean a DailyMed title ('Aspirin (Tablet)' -> 'Aspirin')."""
    return re.split(r"[(\[]", title)[0].strip().title()

# Download and process DailyMed drug labels
logger.info("Fetching DailyMed index...")
index = fetch_drug_index(N_DAILYMED)
logger.info("Got %d labels; downloading + parsing XML...", len(index))

dm_rows: List[Dict[str, Any]] = []
errors = 0
start = time.time() # Issue #11: track throughput + ETA
for i, d in enumerate(index, 1):
    setid, title = d["setid"], d.get("title", "")
    try:
        xr = SESSION.get(
            f"https://dailymed.nlm.nih.gov/dailymed/services/v2/spls/"
            f"{setid}.xml",
            timeout=30)
        xr.raise_for_status()
        root = ET.fromstring(xr.content)
        name = drug_name(title)

```

```

# POSITIVE (label 1): real adverse-reactions text
adverse = extract_section_text(root, LOINC_ADVERSE)
if adverse and len(adverse) > 30:
    dm_rows.append({
        "text": adverse,
        "drug_name": name,
        "group_id": f"dm_{setid}",
        "domain": "Formal (DailyMed)",
        "label": 1,
        "source": "DailyMed",
    })

# NEGATIVE (label 0): real non-adverse text (indications, else
dosage)
neg = (extract_section_text(root, LOINC_INDICATIONS)
      or extract_section_text(root, LOINC_DOSAGE))
if neg and len(neg) > 30:
    dm_rows.append({
        "text": neg,
        "drug_name": name,
        "group_id": f"dm_{setid}",
        "domain": "Formal (DailyMed)",
        "label": 0,
        "source": "DailyMed",
    })
except Exception: # noga: BLE001 - count + continue on any per-
label error
    errors += 1
if i % 25 == 0:
    elapsed = time.time() - start
    rate = i / elapsed if elapsed else 0.0
    eta_min = (len(index) - i) / rate / 60 if rate else 0.0
    logger.info(" %d/%d (rows %d, errors %d, %.1f req/s, ETA
%.0fm) ",
                i, len(index), len(dm_rows), errors, rate, eta_min)
    time.sleep(0.05)

dailymed_df = pd.DataFrame(dm_rows)
logger.info("DailyMed: %d rows | pos %d | neg %d | errors %d",
            len(dailymed_df), int((dailymed_df.label == 1).sum()),
            int((dailymed_df.label == 0).sum()), errors)

```

Extract CADEC(informal domain):

```

# CADEC is already a FOLDER in Google Drive named 'cadec'; mount it.
from google.colab import drive
drive.mount("/content/drive")

```

```

CADEC_DIR = "/content/drive/MyDrive/cadec"

# Issue #7: validate the path AND confine it to Google Drive
assert os.path.isdir(CADEC_DIR), f"Not found: {CADEC_DIR}"
assert os.path.abspath(CADEC_DIR).startswith("/content/drive"), \
    "CADEC_DIR must live inside the mounted Google Drive"
logger.info("Found CADEC folder. Top-level contents: %s",
            sorted(os.listdir(CADEC_DIR))[:15])

def find_folder(base: str, name: str) -> Optional[str]:
    """Return the first subfolder of `base` whose name matches
    `name`."""
    for root, dirs, _ in os.walk(base):
        if os.path.basename(root).lower() == name:
            return root
        for d in dirs:
            if d.lower() == name:
                return os.path.join(root, d)
    return None

TEXT_DIR = find_folder(CADEC_DIR, "text")
ANN_DIR = find_folder(CADEC_DIR, "original")
logger.info("TEXT_DIR: %s", TEXT_DIR)
logger.info("ANN_DIR : %s", ANN_DIR)
assert TEXT_DIR and ANN_DIR, "Could not locate CADEC text/ and
original/ dirs"

```

```

def parse_ann(path: str) -> Tuple[Counter, List[str]]:
    """Parse a BRAT .ann file with validation (Issue #3).

    Returns (entity-type counts, ADR terms). Malformed lines are
    counted
    and logged rather than silently dropped.
    """
    counts: Counter = Counter()
    adr_terms: List[str] = []
    parse_errors = 0
    with open(path, encoding="utf-8", errors="ignore") as f:
        for line in f:
            line = line.strip()
            if not line.startswith("T"):          # text-bound annotations
only
                continue
            p = line.split("\t")
            if len(p) < 2:

```

```

        parse_errors += 1
        continue
    try:
        lab = p[1].split()[0]
        counts[lab] += 1
        if lab == "ADR" and len(p) > 2:
            adr_terms.append(p[2])
    except (IndexError, ValueError):
        parse_errors += 1
if parse_errors:
    logger.warning("%s: %d malformed annotation line(s)",
                  os.path.basename(path), parse_errors)
return counts, adr_terms

# Build CADEC dataset from text + annotation files
cd_rows: List[Dict[str, Any]] = []
for ann in sorted(glob.glob(os.path.join(ANN_DIR, "*.ann"))):
    base = os.path.splitext(os.path.basename(ann))[0]
    txt = os.path.join(TEXT_DIR, base + ".txt")
    if not os.path.exists(txt):
        continue
    with open(txt, encoding="utf-8", errors="ignore") as f:
        text = f.read().strip()
    counts, adr_terms = parse_ann(ann)
    cd_rows.append({
        "text": text,
        "drug_name": "",
        "group_id": f"cadec_{base}",          # post-level group id
        "n_adr": counts.get("ADR", 0),
        "n_symptom": counts.get("Symptom", 0),
        "adr_terms": "; ".join(adr_terms),
        "domain": "Informal (CADEC)",
        "source": "CADEC",
    })

cadec_df = pd.DataFrame(cd_rows)

# Label: 1 if the post has an ADR or Symptom annotation, else 0
cadec_df["label"] = (
    (cadec_df["n_adr"] > 0) | (cadec_df["n_symptom"] > 0)).astype(int)
logger.info("CADEC: %d posts | pos %d | neg %d", len(cadec_df),
            int((cadec_df.label == 1).sum()),
            int((cadec_df.label == 0).sum()))

```

Derive severity and unify the schema:

```
# severity thresholds documented (rationale + caveat)
```

```

SEVERE_KW = ["hospital", "emergency", " er ", "a&e", "ambulance",
"severe",
            "unbearable", "collapsed", "seizure", "anaphyla", "death",
            "fatal", "life-threatening", "swelling", "swelled"]
MODERATE_KW = ["stopped taking", "had to stop", "discontinued", "unable
to",
              "couldn't", "painful", "vomiting", "debilitating",
"interfere"]

def severity_from_text(text: str, label: int, adr_count: int = 0) ->
str:
    """Rule-based severity tier. PROXY only - see severity_source
column."""
    if label == 0:
        return "none"
    t = str(text).lower()
    if any(k in t for k in SEVERE_KW) or adr_count >= 4:
        return "severe"
    if any(k in t for k in MODERATE_KW) or adr_count >= 2:
        return "moderate"
    return "mild"

dailymed_df["severity"] = dailymed_df.apply(
    lambda r: severity_from_text(r["text"], r["label"]), axis=1)
cadec_df["severity"] = cadec_df.apply(
    lambda r: severity_from_text(r["text"], r["label"], r["n_adr"]),
axis=1)

# Tag severity as a derived proxy so it is never mistaken for truth
dailymed_df["severity_source"] = "heuristic_proxy"
cadec_df["severity_source"] = "heuristic_proxy"

keep = ["text", "label", "severity", "severity_source", "domain",
        "group_id", "source"]
dailymed_keep = dailymed_df[keep + ["drug_name"]]
cadec_keep = cadec_df[keep].assign(drug_name="")
df = pd.concat([dailymed_keep, cadec_keep], ignore_index=True)

# Normalised dedup (lowercase, strip urls, collapse whitespace)
def _norm(t: str) -> str:
    t = re.sub(r"http\S+|www\S+", " ", str(t).lower())
    return re.sub(r"\s+", " ", t).strip()

df["_norm"] = df["text"].map(_norm)

```

```

before = len(df)
df = (df.dropna(subset=["text"])
      .drop_duplicates(subset=["_norm"])
      .drop(columns="_norm")
      .reset_index(drop=True))
logger.info("Dropped %d duplicate / near-duplicate rows", before -
len(df))

logger.info("Unified dataset: %s", df.shape)
print("\nBy domain:"); print(df["domain"].value_counts())
print("\nBy label:"); print(df["label"].value_counts())
print("\nBy severity:"); print(df["severity"].value_counts())

```

Preprocess the two domains: formal and informal

```

# WordNetLemmatizer reduces inflected forms to a base ("vomiting" ->
"vomit"),
# improving vocabulary consistency across DailyMed prose and CADEC
posts.
lemm = WordNetLemmatizer()
stop_words = set(stopwords.words("english"))

def clean_text(t: str) -> str:
    """Lightweight structural normalisation - runs BEFORE
tokenisation."""
    t = str(t).lower()
    t = re.sub(r"http\S+|www\S+|\S+\S+", "", t)
    t = re.sub(r"\s+", " ", t).strip()
    return t

def tokenize(t: str) -> List[str]:
    """Convert a cleaned string into lemmatised, filtered tokens."""
    t = re.sub(r"[^a-z0-9\s]", "", t)
    return [lemm.lemmatize(w) for w in t.split()
            if w not in stop_words and len(w) >= 3]

# Apply the preprocessing pipeline. We compute tokens in a TEMP
variable and
# only persist text_processed + word_count_processed

_tokens = df["text"].apply(clean_text).apply(tokenize)
df["text_processed"] = _tokens.apply(" ".join)
df["word_count"] = df["text"].str.split().str.len()
df["word_count_processed"] = df["text_processed"].str.split().str.len()

# Drop rows that became empty after cleaning

```

```

df = df[df["word_count_processed"] > 0].reset_index(drop=True)

# enforce text_processed/word_count_processed consistency
assert (df["text_processed"].str.split().str.len()
        == df["word_count_processed"]).all(), \
        "word_count_processed does not match text_processed"
logger.info("Column consistency check passed")

# Issue #2: report preprocessing compression so we catch over-
aggressive
# stopwords removal (e.g. a long formal doc collapsing to a handful of
tokens)
for dom in df["domain"].unique():
    sub = df[df["domain"] == dom]
    q05 = sub["word_count_processed"].quantile(0.05)
    compression = 1 - (sub["word_count_processed"].sum()
                       / max(sub["word_count"].sum(), 1))
    logger.info("%s | 5th-pct tokens %.0f | mean compression %.0f%%",
                dom, q05, compression * 100)
    if q05 < 20 and "Formal" in dom:
        logger.warning("Formal domain very short after preprocessing -
"
                        "consider tuning the stopwords list.")

logger.info("After preprocessing: %s", df.shape)
df[["domain", "label", "severity", "text", "text_processed"]].head(4)

```

Cross domain EDA:

```

# Split dataset into formal and informal domains
formal = df[df["domain"].str.contains("Formal")]
informal = df[df["domain"].str.contains("Informal")]

def labelBars(ax, fmt: str = "{:.0f}", fontsize: int = 9) -> None:
    """Annotate each vertical bar with its value."""
    for p in ax.patches:
        h = p.get_height()
        if h and not np.isnan(h):
            ax.annotate(fmt.format(h),
                        (p.get_x() + p.get_width() / 2, h),
                        ha="center", va="bottom", fontsize=fontsize,
                        xytext=(0, 2), textcoords="offset points")

def labelHbars(ax, fontsize: int = 9) -> None:
    """Annotate each horizontal bar with its value."""
    for p in ax.patches:
        w = p.get_width()

```

```

        if w and not np.isnan(w):
            ax.annotate(f"{w:.0f}",
                        (w, p.get_y() + p.get_height() / 2),
                        ha="left", va="center", fontsize=fontsize,
                        xytext=(3, 0), textcoords="offset points")

# build vocabularies from text_processed.split() (robust whether the
# frame is in-memory or reloaded from CSV; the old code iterated a
token list
# that becomes a STRING after a CSV round-trip, counting characters).

fvoc = Counter(w for text in formal["text_processed"] for w in
text.split())
ivoc = Counter(w for text in informal["text_processed"] for w in
text.split())
overlap = len(set(fvoc) & set(ivoc))

fig, ax = plt.subplots(2, 3, figsize=(17, 10))
fig.suptitle("Cross-Domain Exploratory Data Analysis",
             fontsize=15, fontweight="bold", y=1.00)

# (1) Overall label balance
counts = df["label"].value_counts().sort_index()
ax[0, 0].bar(["No ADE (0)", "ADE (1)"], counts.values,
             color=["#2E8B57", "#C44E52"])
ax[0, 0].set_title("(1) Overall Label Balance", fontweight="bold")
ax[0, 0].set_xlabel("Adverse Drug Event label")
ax[0, 0].set_ylabel("Number of records")
label_bars(ax[0, 0])

# (2) Label by domain
ct = pd.crosstab(df["domain"], df["label"])
ct.plot(kind="bar", ax=ax[0, 1], color=["#2E8B57", "#C44E52"])
ax[0, 1].set_title("(2) Label Distribution by Domain",
                  fontweight="bold")
ax[0, 1].set_xlabel("Domain")
ax[0, 1].set_ylabel("Number of records")
ax[0, 1].tick_params(axis="x", rotation=15)
ax[0, 1].legend(["No ADE (0)", "ADE (1)", title="Label")
label_bars(ax[0, 1])

# (3) Severity (derived proxy)
order = ["none", "mild", "moderate", "severe"]
sev = df["severity"].value_counts().reindex(order).fillna(0)
ax[0, 2].bar(order, sev.values,
             color=["#2E8B57", "#FFA500", "#FF6347", "#C44E52"])
ax[0, 2].set_title("(3) Severity Distribution (derived proxy)",
                  fontweight="bold")

```

```

ax[0, 2].set_xlabel("Severity level")
ax[0, 2].set_ylabel("Number of records")
label_bars(ax[0, 2])

# (4) Text length by domain
ax[1, 0].hist([formal["word_count"], informal["word_count"]], bins=20,
              label=["Formal (DailyMed)", "Informal (CADEC)"],
              color=["#4C72B0", "#2E8B57"])
ax[1, 0].set_title("(4) Text Length by Domain", fontweight="bold")
ax[1, 0].set_xlabel("Words per record")
ax[1, 0].set_ylabel("Number of records (frequency)")
ax[1, 0].legend(title="Domain")

# (5) Vocabulary overlap
vals = [overlap, len(set(fvoc) - set(ivoc)), len(set(ivoc) -
set(fvoc))]
ax[1, 1].bar(["Shared\n(both domains)", "Formal only", "Informal
only"],
             vals, color=["#FFA500", "#4C72B0", "#2E8B57"])
ax[1, 1].set_title("(5) Vocabulary Overlap Between Domains",
fontweight="bold")
ax[1, 1].set_xlabel("Vocabulary group")
ax[1, 1].set_ylabel("Number of unique tokens")
label_bars(ax[1, 1])

# (6) Top 10 formal-domain terms
topf = fvoc.most_common(10)
if topf:
    words, freqs = zip(*topf)
    ax[1, 2].barh(words, freqs, color="#4C72B0")
    ax[1, 2].invert_yaxis()
    ax[1, 2].set_title("(6) Top 10 Terms - Formal Domain",
fontweight="bold")
    ax[1, 2].set_xlabel("Frequency (token count)")
    ax[1, 2].set_ylabel("Token")
    label_hbars(ax[1, 2])

plt.tight_layout()
plt.savefig("unified_eda.png", dpi=120, bbox_inches="tight")
plt.show()

logger.info("Formal avg words: %.1f | Informal avg words: %.1f",
            formal["word_count"].mean(), informal["word_count"].mean())
logger.info("Vocabulary overlap: %d shared tokens", overlap)

```

Coding for cross domain experiment:

```
!pip install -q numpy pandas scikit-learn xgboost matplotlib

import os
import sys
sys.path.append('.')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import train_test_split
from typing import Callable, Dict, List, Tuple

import eval_utils as ev
import cross_domain as cd

# Project palette
NAVY, RED, GREEN, BLUE, ORANGE = (
    '#00002D', '#C44E52', '#2E8B57', '#4C72B0', '#E69138')

# Experiment configuration
DOMAINS = ('Formal', 'Informal')
TEXT_COL = ev.TEXT_COL          # 'text_processed'
LABEL_COL = ev.LABEL_COL        # 'label'
DOMAIN_COL = ev.DOMAIN_COL      # 'domain'
TEST_SIZE = 0.25
SEED = ev.RANDOM_STATE

# Cross-domain re-splits by DOMAIN, so we use the pooled frame.
_, _, _, full_df = ev.load_splits(
    'train.csv', 'val.csv', 'test.csv', 'unified_full.csv')
print('Pooled rows:', len(full_df))
print('Domain counts:',
      full_df[DOMAIN_COL].value_counts().to_dict())
def domain_mask(series: pd.Series, name: str) -> pd.Series:
    """
    Match a short domain name against full labels.

    'Formal' matches 'Formal (DailyMed)'. Case-insensitive on the
    prefix.
    """
    return series.astype(str).str.strip().str.lower().str.startswith(
        name.strip().lower())
```

```

def make_domain_splits(df: pd.DataFrame, domains: Tuple[str, ...],
                      domain_col: str, label_col: str,
                      test_size: float = 0.25,
                      seed: int = 42) -> Dict[str, Tuple]:
    """
    Stratified within-domain train/test split for each domain.

    Splitting *within* each domain is what makes the experiment
    leakage-free:
    in-domain performance is scored on held-out rows (disjoint from
    training),
    and cross-domain uses an entirely different domain - so no row is
    ever
    both trained on and evaluated.

    Returns

    {domain: (train_df, test_df)} with reset indices.
    """
    splits: Dict[str, Tuple] = {}
    for dom in domains:
        sub = df[df[domain_col] == dom].reset_index(drop=True)
        train_df, test_df = train_test_split(
            sub, test_size=test_size, random_state=seed,
            stratify=sub[label_col])
        splits[dom] = (train_df.reset_index(drop=True),
                       test_df.reset_index(drop=True))
    return splits


def run_cross_domain_matrix(splits: Dict[str, Tuple],
                            domains: Tuple[str, ...],
                            build_model: Callable,
                            predict_proba: Callable,
                            label_col: str) -> Tuple[pd.DataFrame,
Dict]:
    """
    Train once per domain; evaluate across every target domain's TEST
    split.

    Each model is trained only on its source domain's TRAIN split and
    scored
    on each target domain's held-out TEST split, so there is no leakage
    on the
    diagonal (held-out) or off-diagonal (different domain). Training
    once per
    domain also avoids re-fitting the same model for every matrix cell.

```

```

Parameters
-----
build_model : (domain, train_df) -> model
predict_proba : (model, df) -> array of P(ADE)

Returns
-----
(results_df, preds) - same schema as
cross_domain.run_domain_matrix, so the
existing grid / degradation / compare helpers work on it unchanged.
"""

models = {dom: build_model(dom, splits[dom][0]) for dom in domains}
rows: List[Dict] = []
preds: Dict[Tuple[str, str], Dict] = {}
for src in domains:
    for tgt in domains:
        test_df = splits[tgt][1]
        y_true = test_df[label_col].values
        proba = np.asarray(
            predict_proba(models[src], test_df)).reshape(-1)
        y_pred = (proba >= 0.5).astype(int)
        metrics = ev.compute_metrics(y_true, y_pred, proba)
        rows.append({"train_domain": src, "test_domain": tgt,
                    "in_domain": src == tgt, **metrics})
        preds[(src, tgt)] = {"y_true": y_true, "y_pred": y_pred,
                            "y_proba": proba}

return pd.DataFrame(rows), preds


def vocabulary_overlap(df: pd.DataFrame, domains: Tuple[str, ...],
                      domain_col: str, text_col: str,
                      top_n: int = 300) -> Dict[str, int]:
    """Jaccard-style overlap of each domain's top TF-IDF terms.

    Explains *why* cross-domain is hard: if the two domains share few
top
terms, a lexical (TF-IDF) model trained on one cannot read the
other.
    """
    from sklearn.feature_extraction.text import TfidfVectorizer

    top_terms = {}
    for dom in domains:
        sub = df[domain_mask(df[domain_col], dom)][text_col].fillna("")
        vec = TfidfVectorizer(max_features=top_n, ngram_range=(1, 1),
                              min_df=2)
        vec.fit(sub.astype(str))
        top_terms[dom] = set(vec.get_feature_names_out())
    a, b = top_terms[domains[0]], top_terms[domains[1]]

```

```

        shared = a & b
        return {"a_only": len(a - b), "shared": len(shared),
                "b_only": len(b - a),
                "jaccard": round(len(shared) / len(a | b), 3) if (a | b)
else 0.0}

splits = make_domain_splits(full_df, DOMAINS, DOMAIN_COL, LABEL_COL,
                             test_size=TEST_SIZE, seed=SEED)

for dom in DOMAINS:
    tr, te = splits[dom]
    print(f'{dom:9s} train={len(tr):4d} test={len(te):4d} '
          f'(test ADE rate {te[LABEL_COL].mean():.2f})')
# C1a: binary F1 grid
table_f1 = cd.combine_model_matrices(all_results, metric='f1')
order = [c for c in ['F->F', 'I->I', 'F->I', 'I->F'] if c in
table_f1.columns]
table_f1 = table_f1[order]
print('=== BINARY F1 (report Table 4a) ===')
display(table_f1.round(4))
table_f1.to_csv('cd_matrix_binary_f1.csv')

# C1b: macro-F1 grid (
try:
    table_macro = cd.combine_model_matrices(all_results,
metric='f1_macro')
    table_macro = table_macro[[c for c in order if c in
table_macro.columns]]
    print('\n=== MACRO-F1 (report Table 4) ===')
    display(table_macro.round(4))
    table_macro.to_csv('cd_matrix_f1_macro.csv')
except Exception as exc: # noqa: BLE001
    print('macro_f1 grid unavailable from combine_model_matrices:',
exc)

    print('Falling back to building it directly from all_results...')
    rows = {}
    for name, res in all_results.items():
        r = {}
        for _, row in res.iterrows():
            key = f"{row['train_domain'][0]}->{row['test_domain'][0]}"
            r[key] = row.get('f1_macro', float('nan'))
        rows[name] = r
    table_macro = pd.DataFrame(rows).T[order]
    display(table_macro.round(4))
    table_macro.to_csv('cd_matrix_f1_macro.csv')

# --- heatmap on the macro grid -----
-----
fig, ax = plt.subplots(figsize=(7.5, 1.4 + 0.55 * len(table_macro)))

```

```

im = ax.imshow(table_macro.values, cmap='RdYlGn', vmin=0.0, vmax=1.0,
                aspect='auto')
ax.set_xticks(range(len(table_macro.columns)))
ax.set_xticklabels(table_macro.columns, fontweight='bold')
ax.set_yticks(range(len(table_macro.index)))
ax.set_yticklabels(table_macro.index, fontweight='bold')
for i in range(len(table_macro.index)):
    for j in range(len(table_macro.columns)):
        val = table_macro.values[i, j]
        ax.text(j, i, f'{val:.3f}', ha='center', va='center',
                fontweight='bold', color='white' if val < 0.6 else
NAVY)
ax.axvline(1.5, color=NAVY, lw=2)
ax.set_title('Model x Domain-Transfer Macro-F1 '
            '(left = in-domain, right = cross-domain)',
            fontweight='bold')
fig.colorbar(im, fraction=0.046, pad=0.04, label='Macro-F1')
plt.tight_layout()
plt.savefig('cd_model_matrix.png', dpi=200, bbox_inches='tight')
plt.show()

```

```

def gap_summary(all_results, metric):
    """Mean in-domain vs mean cross-domain score, with relative
    drop."""
    rows = []
    for name, res in all_results.items():
        if metric not in res.columns:
            continue
        in_d = res[res['in_domain']][metric].mean()
        cross = res[~res['in_domain']][metric].mean()
        rows.append({'model': name,
                    f'in_domain_{metric}': in_d,
                    f'cross_domain_{metric}': cross,
                    'drop_pct': (in_d - cross) / in_d * 100 if in_d
else 0.0})
    return (pd.DataFrame(rows)
            .sort_values(f'cross_domain_{metric}', ascending=False)
            .reset_index(drop=True))

def plot_gap(df, metric, fname):
    x = np.arange(len(df))
    w = 0.38
    fig, ax = plt.subplots(figsize=(max(8, 1.7 * len(df)), 5))
    ax.bar(x - w / 2, df[f'in_domain_{metric}'], width=w,
           label='In-domain', color=GREEN, edgecolor=NAVY)
    ax.bar(x + w / 2, df[f'cross_domain_{metric}'], width=w,

```

```

        label='Cross-domain', color=RED, edgecolor=NAVY)
ax.set_xticks(x)
ax.set_xticklabels(df['model'], rotation=15)
ax.set_xlabel('Model', fontweight='bold')
label = 'Macro-F1' if metric == 'f1_macro' else 'Binary F1'
ax.set_ylabel(label, fontweight='bold')          # label now matches
the data
ax.set_ylim(0, 1.15)
ax.set_title(f'Generalisation Gap by Model - In-Domain vs Cross-
Domain ')

        f'({label})', fontweight='bold', pad=18)
ax.legend(title='Evaluation', loc='upper right')
for i, row in df.iterrows():
    top = max(row[f'in_domain_{metric}'],
row[f'cross_domain_{metric}'])
    ax.annotate(f"-{row['drop_pct']:.0f}%", (i, top + 0.03),
                ha='center', fontweight='bold', color=NAVY)
plt.tight_layout()
plt.savefig(fname, dpi=200, bbox_inches='tight')
plt.show()

summary_f1 = gap_summary(all_results, 'f1')
print('=== Generalisation gap on BINARY F1 ===')
display(summary_f1.round(3))
summary_f1.to_csv('cd_summary_binary_f1.csv', index=False)
plot_gap(summary_f1, 'f1', 'cd_gap_by_model_binary_f1.png')

summary_macro = gap_summary(all_results, 'f1_macro')
if len(summary_macro):
    print('\n=== Generalisation gap on MACRO-F1 (report Table 5) ===')
    display(summary_macro.round(3))
    summary_macro.to_csv('cd_summary_f1_macro.csv', index=False)
    plot_gap(summary_macro, 'f1_macro', 'cd_gap_by_model_f1_macro.png')
else:
    print('\nmacro_f1 not present in all_results - check
ev.compute_metrics.')

# Keep the old name alive for any later cell that expects it
summary_df = summary_f1

```

Confusion matrices for the best model:

```

best_name = summary_df.iloc[0]['model']
best_preds = all_preds.get(best_name)

if best_preds is None:

```

```

    print(f'No stored predictions for {best_name}.')
else:
    pairs = [(s, t) for s in DOMAINS for t in DOMAINS if s != t]
    fig, axes = plt.subplots(1, len(pairs), figsize=(5.2 * len(pairs),
4.5))
    axes = np.atleast_1d(axes)
    for ax, (src, tgt) in zip(axes, pairs):
        cmat = confusion_matrix(best_preds[(src, tgt)]['y_true'],
                                best_preds[(src, tgt)]['y_pred'])
        ax.imshow(cmat, cmap='Blues')
        ax.set_xticks([0, 1])
        ax.set_xticklabels(['No-ADE', 'ADE'])
        ax.set_yticks([0, 1])
        ax.set_yticklabels(['No-ADE', 'ADE'])
        ax.set_xlabel('Predicted')
        ax.set_ylabel('True')
        ax.set_title(f'{best_name}: {src} -> {tgt}',
fontweight='bold')
        thr = cmat.max() / 2 if cmat.max() else 0.5
        for i in range(cmat.shape[0]):
            for j in range(cmat.shape[1]):
                ax.text(j, i, cmat[i, j], ha='center', va='center',
                        fontweight='bold',
                        color='white' if cmat[i, j] > thr else NAVY)
        fig.suptitle('Cross-Domain Confusion - Best Model',
fontweight='bold')
    plt.tight_layout()
    plt.savefig('cd_confusion.png', dpi=200, bbox_inches='tight')
    plt.show()

```

ROC-AUC for the domains:

```

fi = diag[diag['transfer'] == 'F->I'].sort_values('macro_f1_corrected',
                                                    ascending=False)

x = np.arange(len(fi))
w = 0.38
fig, ax = plt.subplots(figsize=(max(8, 1.7 * len(fi)), 5))
ax.bar(x - w / 2, fi['macro_f1@0.5'], width=w,
        label='At default threshold (0.5)', color=RED, edgecolor=NAVY)
ax.bar(x + w / 2, fi['macro_f1_corrected'], width=w,
        label='After prior correction', color=GREEN, edgecolor=NAVY)
ax.set_xticks(x)
ax.set_xticklabels(fi['model'], rotation=15)
ax.set_ylabel('Macro-F1', fontweight='bold')
ax.set_xlabel('Model', fontweight='bold')
ax.set_ylim(0, 1.0)
ax.set_title('Formal -> Informal transfer: label-free prior
correction',

```

```

        fontweight='bold', pad=14)
ax.legend()
for i, (_, r) in enumerate(fi.iterrows()):
    ax.annotate(f"AUC {r['roc_auc']:.3f}",
                (i, max(r['macro_f1@0.5'], r['macro_f1_corrected']) +
0.03),
                ha='center', fontsize=9, color=NAVY)
plt.tight_layout()
plt.savefig('cd_prior_correction.png', dpi=200, bbox_inches='tight')
plt.show()

```

Coding for the explainability:

```

# Run ONCE per session; installs the packages the 6 modules needed
!pip install -q pandas numpy scikit-learn transformers torch lime shap
faiss-cpu plotly
# Classic kaleido bundles its own Chromium
!pip install -q "kaleido==0.2.1"

# Silence the ipywidgets progress bars
import os
os.environ["HF_HUB_DISABLE_PROGRESS_BARS"] = "1"
os.environ["TRANSFORMERS_NO_ADVISORY_WARNINGS"] = "1"
try:
    import transformers
    transformers.logging.set_verbosity_error()
    transformers.utils.logging.disable_progress_bar()
except Exception:
    pass

%matplotlib inline

print('Setup ready – now run the cells below in order.')

```

Setup

```

!pip install -q transformers torch lime shap faiss-cpu scikit-learn
pandas
# import sys; sys.path.append('src')

import pandas as pd
import eval_utils as ev
import explainability as ex
import case_retrieval as cr
import ade_agent as ag

```

```
import biobert_infer as bb

MODEL_DIR = 'biobert_out'      # <-- your saved fine-tuned BioBERT
directory
train_df, val_df, test_df, full_df = ev.load_splits(
    'train.csv', 'val.csv', 'test.csv', 'unified_full.csv')
```

Load fine tuned Biobert

```
# Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')

# Point at your actual folder (you said it's called biobert_model, on
Drive)
MODEL_DIR = '/content/drive/MyDrive/biobert_model'

# Verify it's really there and contains a saved model BEFORE loading
import os
assert os.path.isdir(MODEL_DIR), f"Folder not found: {MODEL_DIR}"
print("Folder contents:", os.listdir(MODEL_DIR))
```

```
biobert = bb.load_biobert(MODEL_DIR, max_len=128)
print('Loaded BioBERT on', biobert.device)

# sanity check: a probability for one example
sample = test_df[ev.RAW_TEXT_COL].iloc[0]
print('P(ADE) =', round(biobert.classify_fn(sample), 3))
```

Lime on BioBert :

```
lime_words = ex.explain_with_lime(
    sample, biobert.predict_proba, num_features=8, num_samples=200)
print('LIME drivers (BioBERT):')
for w, wt in lime_words:
    print(f' {w:20s} {wt:+.3f}')
```

Shap on Biobert:

```
shap_words = ex.shap_top_words_for_text(
    sample, biobert.predict_proba, top_k=8, max_evals=120)
print('SHAP drivers (BioBERT):')
for w, wt in shap_words:
    print(f' {w:20s} {wt:+.4f}')
```

Faiss case retrieval in Biobert embedding space:

```
retriever =  
cr.CaseRetriever(bb.biobert_embedder(biobert)).build(train_df)  
neighbours = retriever.query(sample, k=10)  
print(cr.format_neighbours(neighbours))
```

Agent on Biobert(Lime + Shap together)

```
severity_fn = lambda t: ('severe' if any(w in t.lower() for w in  
['severe', 'emergency', 'hospital', 'breathing', 'swelling', 'collapsed'])  
    else 'moderate' if any(w in t.lower() for w in  
['stopped', 'vomiting', 'unable'])  
    else 'mild')  
  
agent = ag.build_agent(  
    classify_fn=biobert.classify_fn,  
    retriever=retriever,  
    explain_fn=lambda t: ex.explain_with_lime(  
        t, biobert.predict_proba, num_features=6, num_samples=150),  
    shap_fn=lambda t: ex.shap_top_words_for_text(  
        t, biobert.predict_proba, top_k=6, max_evals=100),  
    severity_fn=severity_fn,  
    confidence_margin=0.15)  
  
result = agent.run(sample)  
print(ag.format_trace(result))  
print('\nLIME/SHAP agreement:',  
result.evidence.get('lime_shap_agreement'))
```

Running a clinical triage case:

```
# !pip install -q plotly  
import xai_visuals as xv  
import eval_utils as ev  
from IPython.display import HTML, display  
  
# Make every Plotly `.show()` below render as a static PNG so the  
figures  
xv.use_static_renderer(scale=2)
```

```
# Pick a RANDOM real case from the held-out test set.
positive_only = True
RANDOM_STATE = 42

pool = test_df[test_df[ev.LABEL_COL] == 1] if positive_only else
test_df
chosen = pool.sample(n=1, random_state=RANDOM_STATE).iloc[0]

trriage_case = chosen[ev.RAW_TEXT_COL]
true_label = 'ADE' if chosen[ev.LABEL_COL] == 1 else 'No ADE'
true_domain = chosen.get(ev.DOMAIN_COL, 'unknown')

print('Randomly selected clinical-triage case (held-out test set)')
print('Domain      :', true_domain)
print('True label  :', true_label)
print('Case text   :\n', tririage_case)
```

Biobert prediction + gauge:

```
proba = biobert.classify_fn(trriage_case)
severity = severity_fn(trriage_case)
print(f'BioBERT P(ADE) = {proba:.3f} | severity = {severity}')

xv.trriage_gauge(proba, severity, 'Triage confidence - clinical
case').show()
```

Pick a formal and informal ADE case

```
import xai_visuals as xv
import eval_utils as ev

# ADE-positive cases from each domain
formal_pool = test_df[(test_df[ev.LABEL_COL] == 1) &
                      (test_df[ev.DOMAIN_COL].str.contains('Formal',
case=False, na=False))]
informal_pool = test_df[(test_df[ev.LABEL_COL] == 1) &
                        (test_df[ev.DOMAIN_COL].str.contains('Informal', case=False,
na=False))]

# random_state fixed so the report figure is reproducible; set to None
for fresh picks
formal_case = formal_pool.sample(1,
random_state=42).iloc[0][ev.RAW_TEXT_COL]
informal_case = informal_pool.sample(1,
random_state=42).iloc[0][ev.RAW_TEXT_COL]
```

```
print('FORMAL (DailyMed):\n', formal_case[:300], '\n')
print('INFORMAL (CADEC):\n', informal_case[:300])
```

Explain with Biobert:

```
formal_words = ex.explain_with_lime(
    formal_case, biobert.predict_proba, num_features=8,
    num_samples=200)
informal_words = ex.explain_with_lime(
    informal_case, biobert.predict_proba, num_features=8,
    num_samples=200)

print('Formal drivers  :', [w for w, _ in formal_words])
print('Informal drivers:', [w for w, _ in informal_words])
```

Side by Side comparison:

```
xv.lime_vs_shap(formal_words, informal_words).show()
```

Faiss: similar know cases (precedent)

```
neighbours = retriever.query(triage_case, k=10)
xv.similar_cases_table(neighbours).show()
```

The full, runnable code is in the team GitHub repository (link on the front page) with a README describing setup and execution. The core Python modules are:

- `eval_utils.py`: shared metric definitions (accuracy, precision, recall, F1, macro-F1, ROC-AUC), cross-domain helpers and split loaders used by every model tier.
- `baseline_models.py`: TF-IDF vectorisation and the logistic-regression, random-forest and XGBoost baselines.
- `bilstm_attention.py`: bidirectional LSTM with attention (embedding → Bi-LSTM → attention → dense sigmoid).
- `svm_bert.py`: BERT-embedding extraction with an RBF-kernel SVM classifier.
- `biobert_finetune.py` and `biobert_infer.py`: BioBERT fine-tuning and the inference adapter exposing `predict_proba`, `classify_fn` and `embed`.
- `ensemble.py`: stacked logistic-regression meta-learner over uid-keyed base-model probabilities, with a column-order mismatch guard.
- `retrieval_pipeline.py`: FAISS case retrieval in BioBERT embedding space.
- `xai_visuals.py`: LIME/SHAP visuals, agreement indicator, triage gauge and precedents table.

- agent.py: the triage agent with a genuine decision loop (confident cases skip tools; uncertain/severe cases invoke them and escalate).
- app.py (Streamlit) and the Flask safety_net_app: the interactive application with consent gate, role separation and audit logging.
- tests/: fourteen pytest cases (metrics, ensemble merge/guard, retrieval, inference adapters, agent routing) run in CI via GitHub Actions on every push.

The notebooks (baselines, Bi-LSTM, SVM-BERT, BioBERT, explainability/FAISS, and the merged cross-domain experiment) contain the executed outputs the metric tables and figures reproduced in this report.

Appendix B — Provenance

Corpus pull timestamp 16 June 2026; random seed 42; DailyMed requested n = 1,500 (April 2026 release); total harmonised records 3,077 (formal 1,831, informal 1,246; positives 1,853, negatives 1,224). Positive formal evidence LOINC 34084-4; negative formal evidence LOINC 34067-9 and 34068-7. Severity is a derived heuristic proxy, not a ground-truth clinical label.

Component	Parameter	Value	Rationale
Global	random_state / seed	42	Reproducibility across all tiers
Global	domain split	75 / 25 stratified	Leakage-free per-domain split
Global	pooled split	70 / 15 / 15 stratified	Train / val / test for deployment models
TF-IDF	ngram_range	(1, 2)	Unigrams and bigrams; captures short negation cues
TF-IDF	min_df	2	Drops hapax terms that cannot generalise
TF-IDF	max_features	20,000	Bounds dimensionality against 3,077 records
TF-IDF	lowercase / strip	True	Applied in pre-processing (Section 2.4)
LogReg	max_iter	1000	Ensures convergence on sparse input
LogReg	class_weight	balanced	Counters 60.2% positive prior
LogReg	C, penalty, solver	defaults (1.0, L2, lbfgs)	Not tuned
Random forest	n_estimators	200	Variance reduction without excess cost
Random forest	class_weight	balanced	Counters class imbalance
Random forest	max_depth	None (default)	Not tuned
XGBoost	n_estimators	300	Set by convention
XGBoost	max_depth	6	Set by convention
XGBoost	eval_metric	logloss	Probability-calibrated objective
XGBoost	learning_rate	default (0.3)	Not tuned
Bi-LSTM	architecture	Bi-LSTM + attention	See Section 3.3
Bi-LSTM	vocabulary size	len(vocab)	[from bilstm_attention.py]
Bi-LSTM	max sequence length	20000	
SVM-BERT	encoder	bert-base (frozen)	Embeddings only; no fine-tuning
SVM-BERT	pooling	mean	[from svm_bert.py]
BioBERT	checkpoint	dmis-lab/biobert-base-cased-v1.1	Biomedical pre-training
BioBERT	epochs	3	Standard for small-corpus fine-tuning
BioBERT	max sequence length	128	Set in biobert_infer.load_biobert()

BioBERT	internal validation split	0.15 stratified	Cross-domain runs; monitors unseen data
Ensemble	base models	Bi-LSTM, SVM-BERT, BioBERT	Set in ensemble.BASE_MODELS
Ensemble	meta-learner	LogisticRegression	Stacking on val-fold probabilities
Ensemble	meta_max_iter	1000	Convergence
Ensemble	meta_class_weight	balanced	Counters class imbalance

Hyperparameters table for this project

Appendix C — Team roles and project management

The project ran over a ten-week schedule (15 May – 23 July 2026) in one-week Agile sprints, with four milestones: baseline established (M1, 29 May), all models complete (M2, 19 June), cross-domain results (M3, 3 July) and application ready (M4, 17 July). Work was tracked as GitHub issues, merged through peer-reviewed pull requests, and gated by continuous integration. Roles were:

- Francis (Chisom Ofonagoro) — Team Leader: agentic framework and overall direction; Bi-LSTM and BioBERT modelling.
- Hassan — Scrum Master: Agile ceremonies; SVM–BERT hybrid and Streamlit integration.
- Sai Charan Ganagani (Charan) — GitHub Manager: repository and branching; data ingestion, loaders, splits and baseline scripts.
- Sam — Communications Lead: data-management planning and communications; BioBERT fine-tuning and FAISS retrieval.
- Renjie (Jay) — Code Review Manager: pipeline documentation and pull-request review; Streamlit application front end.
- Deepak — Test Plan Manager: automated test plan and coverage; Bi-LSTM and ensemble validation.

Appendix D — Tables:

Model	F→F	I→I	F→I	I→F
TF-IDF + LogReg	0.9972	0.9597	0.0000	0.5710
TF-IDF + RF	1.0000	0.9619	0.0000	0.6819
TF-IDF + XGB	1.0000	0.9242	0.0140	0.7532
Bi-LSTM + attention	1.0000	0.9615	0.0000	0.5964
SVM-BERT	0.9916	0.9724	0.0546	0.5647
BioBERT	1.0000	0.9771	0.0412	0.5831

Table 4a. The same transfers on binary F1 (positive class = ADE). Three models score exactly zero on Formal→Informal: they predict no positive case at all on a target that is 91.0% positive. This is invisible in macro-F1, which still credits those models 0.0824 for labelling the negative class correctly.

Model	Transfer	Pred. pos. rate @0.5	ROC-AUC	Balanced acc. @0.5	Macro-F1 @0.5	Macro-F1 corrected
TF-IDF + LogReg	F→I	0.000	0.654	0.500	0.0824	0.5292
TF-IDF + RF	F→I	0.000	0.571	0.500	0.0824	0.4765
TF-IDF + XGB	F→I	0.006	0.593	0.504	0.0898	0.4756
Bi-LSTM + attention	F→I	0.000	0.544	0.500	0.0824	0.4899
SVM-BERT	F→I	0.029	0.664	0.496	0.1089	0.5292
BioBERT	F→I	0.022	0.492	0.493	0.1017	0.5684
TF-IDF + LogReg	I→F	0.993	0.922	0.506	0.2963	0.8214

TF-IDF + RF	I→F	0.754	0.843	0.695	0.6252	0.7664
TF-IDF + XGB	I→F	0.616	0.912	0.786	0.7500	0.8080
Bi-LSTM + attention	I→F	0.934	0.995	0.555	0.3965	0.9908
SVM-BERT	I→F	0.993	0.451	0.496	0.2859	0.4686
BioBERT	I→F	0.950	0.689	0.533	0.3620	0.6244

Table 4b. Cross-domain diagnostics. Target prevalence is 91.0% for the informal test partition (n = 312) and 39.7% for the formal one (n = 456). "Pred. pos. rate" is the fraction of the target predicted ADE at the default 0.5 threshold. "Macro-F1 corrected" applies the label-free prior correction described in Section 4.2.1.